

Neo4j JDBC Driver

2026-05-21T07:40:47Z

Table of Contents

1. Introduction	1
1.1. Features	1
1.2. Limitations	2
1.3. When to use the Neo4j JDBC Driver?	2
1.4. Differences with the previous versions of this driver and other JDBC drivers for Neo4j	3
2. Usage	3
2.1. Getting a connection via environment variables	6
3. Distribution	7
3.1. Driver modules and dependencies	7
3.2. Available bundles	8
4. Configuration	10
4.1. Driver class name	10
4.2. URL and connection properties	10
4.3. Getting a driver or a connection instance	17
4.4. Securing your connection by using SSL	18
4.5. Causal clustering and bookmarks	19
4.6. Neo4j transactional metadata	20
4.7. Logging	21
4.8. Metrics	23
4.9. Tracing	24
4.10. Custom authentication	29
5. Metadata	35
5.1. Catalog and schema	36
5.2. Labels to tables	37
5.3. Primary keys	37
6. SQL to Cypher translation	37
6.1. Introduction	37
6.2. Translating SQL to Cypher	38
6.3. Using the default translator	40
7. Cypher-backed views	75
7.1. Introduction	75
7.2. Requirements	75
7.3. Configuration and view definition format	76
7.4. Restrictions	79
8. Object mapping	79
8.1. Introduction	79
9. text2cypher	83
Appendix A: Neo4j specific conversions	85

A.1. Data types for fixed-point arithmetic	85
A.2. SQL Date, Time and Timestamps	85
A.3. Vector support	86
Appendix B: Neo4j specific syntax for statements	87
B.1. Callable statements	87
Appendix C: Migrating from older versions or other JDBC drivers for Neo4j	88
C.1. Migrating from version 4 or 5 of this driver	88

Abstract

This is the manual for the official Neo4j JDBC Driver.

*This driver is officially supported and endorsed by Neo4j. It is a standalone driver, independent of and **not** built on top of the [common Neo4j Java Driver](#). While the latter provides a Neo4j-idiomatic way to access Neo4j from Java, the JDBC driver adheres to [JDBC 4.3](#).*



*This documentation refers to **this** driver as the Neo4j JDBC Driver and to the idiomatic Neo4j driver as the common Neo4j Java Driver.*

1. Introduction

JDBC stands for "Java Database Connectivity" and is thus not bound exclusively to relational databases. Nevertheless, JDBC's terms, definitions, and behavior are highly influenced by SQL and relational databases. As Neo4j is a graph database with quite a different paradigm than relational and a non-standardized behaviour in some areas, there might be some details that don't map 100% in each place, and we make sure to educate you about these in this documentation.

This documentation focuses on install, use, and configure the Neo4j JDBC Driver, as well as discussing the driver's design choices. While we do provide runnable examples showing how to use JDBC with Neo4j, this is not a documentation about how to correctly use JDBC as an API.



The Neo4j JDBC Driver requires JDK 17 on the client side and Neo4j 5.5+ on the server side. To use it with a Neo4j cluster, server-side routing must be enabled on the cluster.

1.1. Features

- Fully supports the Java module system
- Adheres to JDBC 4.3
- Can run any Cypher statement
- Implements `DatabaseMetaData` and `ResultSetMetaData` as fully as possible with a nearly schemaless database and general very flexible result sets, allowing for automatic metadata retrieval from ETL and ELT tools
- Provides an [SPI](#) to hook in translators from SQL to Cypher
- Provides an optional default implementation to translate many SQL statements into semantically similar Cypher statements

- Supports client-side Cypher-backed views
- Can be safely used with JDBC connection pools as opposed to the common Neo4j Java Driver or any JDBC driver based on that, as it doesn't do internal connection pooling and transaction management otherwise than dictated by the JDBC Spec
- Built-in token based authentication, including reauthentication on token expiration plus an optional Keycloak based SSO module
- Built-in JSON based Object mapping

The absence of any connection pooling and transaction management is an advantage of the Neo4j JDBC Driver over the common Neo4j Java Driver. It allows to pick and choose any database connection pooling system such as [HikariCP](#) and transaction management such as [Jakarta Transactions](#).

1.2. Limitations

- The database metadata is retrieved using Neo4j's schema methods, such as `db.labels`, `db.schema.nodeTypeProperties()`, which may not always be accurate
- While single label nodes map naturally to table names, nodes with multiple labels don't
- There is no reliable way to always determine the datatype for properties on nodes, as it would require reading all of them (which this driver does not do)
- Some JDBC features are not supported yet (such as the `CallableStatement`); some feature will never be supported
- The SQL to Cypher translator supports only a limited subset of clauses and SQL constructs that can be equivalently translated to Cypher (See [Supported statements](#))
- There is no "right" way to map `JOIN` statements to relationships, so your mileage may vary

1.3. When to use the Neo4j JDBC Driver?

- Integration with ETL and ELT tools that don't offer an integration based on the common Neo4j Java driver
- An easier on-ramp towards Neo4j for people familiar with JDBC, who want to keep using that API, but with Cypher and Neo4j
- Integration for ecosystems like Jakarta EE whose transaction management directly supports any JDBC-compliant driver
- Integration with database migration tools such as Flyway

There is no need to redesign an application that is built on the common Neo4j Java Driver to migrate to this driver. If your ecosystem already provides a higher-level integration based on the common Neo4j Java Driver, such as [Spring Data Neo4j \(SDN\)](#) for [Spring](#), there is no need to switch to something else. In case of [Quarkus](#), the Neo4j JDBC Driver is an option to consider: although we do provide an integration for the [common Neo4j Java Driver](#), this integration does not support Quarkus' transaction systems in contrast to this driver.

As there is little incentive to use this driver with Hibernate ([Neo4j-OGM](#) or SDN are the best alternatives for Neo4j), it might be worth giving [Spring Data JDBC](#) a try.

1.4. Differences with the previous versions of this driver and other JDBC drivers for Neo4j

Several other JDBC drivers exists for Neo4j, most notably the previous versions 4 and 5 of this driver. Most (if not all) of them wrap the common Neo4j Java Driver and implement the JDBC spec on top of that. This comes with a number of issues:

- You end up with a *pool of connection pools*, because the common Neo4j Java Driver manages a connection pool, whereas JDBC drivers delegate this task to dedicated pooling solutions.
- The transaction management of the common Neo4j Java Driver is not aligned with the way JDBC manages transactions.
- Older versions of the Neo4j JDBC driver shade a few dependencies, such as [Jackson](#) as well as additional logging frameworks. This takes a toll on the classpath and, in case of logging, it leads to runtime problems.
- Existing drivers with an SQL-to-Cypher translation layer are "read-only" and don't support write statements, so they cannot be used for ETL use-cases aiming to ingest data into Neo4j.



This driver does not support automatic reshaping or flattening of the result sets, as the previous versions do. If you query for nodes, relationships, paths, or maps, you should use `getObject` on the result sets and cast them to the appropriate type (you find all of them inside the package `org.neo4j.jdbc.values`). However, the default SQL-to-Cypher translator will (when connected to a database) figure out what properties nodes have and turn the asterisk (*) into individual columns of nodes and relationships, just like what you would expect when running a `SELECT *` statement.

For information on upgrade/migration from other drivers to this one, see [\[[^]top\]](#).

2. Usage

Add the JDBC driver to your application, for example as a Gradle dependency:

Listing 1. Using the full bundle as a runtime dependency inside a Gradle-based project

```
dependencies {  
    runtimeOnly(org.neo4j:neo4j-jdbc-full-bundle:6.12.3) ①  
}
```

① The coordinates are the same for a Maven project.

You can then use the Neo4j JDBC driver as you would do with any other JDBC driver:



In case any tooling asks you for the name of the concrete driver class, it is

Listing 2. Acquire a connection and execute a query

```
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.List;

public final class Quickstart {

    static void queryWithCypher() {
        var query = """
            MATCH (m:Movie)<-[:ACTED_IN]-(p:Person)
            RETURN m.title AS title, collect(p.name) AS actors
            ORDER BY m.title
            """;

        var url = "jdbc:neo4j://localhost:7687/movies";
        var username = "neo4j";
        var password = "verysecret";

        try (var con = DriverManager.getConnection(url, username, password); ❶
            var stmt = con.createStatement(); ❷
            var result = stmt.executeQuery(query)) { ❸

            while (result.next()) { ❹
                var movie = result.getString(1); ❺
                var actors = (List<String>) result.getObject("actors"); ❻
                System.out.printf("%s\n", movie);
                actors.forEach(actor -> System.out.printf("\t * %s\n", actor));
            }
        } catch (SQLException ex) {
            throw new RuntimeException(ex);
        }
    }
}
```

- ❶ Instantiate a JDBC connection. There's no need to do any class loading beforehand, the driver will be automatically registered
- ❷ Create a (reusable) statement
- ❸ Execute a query
- ❹ Iterate over the results, as with any other JDBC result set
- ❺ JDBC's indexing starts at 1
- ❻ JDBC also allows retrieval of result columns by name; the Neo4j JDBC driver also supports complex objects, such as lists

In the example above we used Neo4j's *lingua franca*, [Cypher](#), to query the database. The Neo4j JDBC

Driver has limited support for using SQL as well. It can do so automatically or on a case-by-case basis. To translate a single, call `java.sql.Connection#nativeSQL(String)` and use the result in your queries. For automatic translation, instantiate the driver setting the optional URL parameter `sql2cypher` to `true`. The following example shows how:

Listing 3. Configure the JDBC driver to automatically translate SQL to cypher.

```
var query = """
    SELECT m.title AS title, collect(p.name) AS actors
    FROM Person as p
    JOIN Movie as m ON (m.id = p.ACTED_IN)
    ORDER BY m.title
    """; ①

var url = "jdbc:neo4j://localhost:7687/movies?enableSQLTranslation=true";

try (var con = DriverManager.getConnection(url, username, password);
    var stmt = con.createStatement();
    var result = stmt.executeQuery(query)) {

    while (result.next()) {
        var movie = result.getString(1);
        var actors = (List<String>) result.getObject("actors");
        System.out.printf("%s\n", movie);
        actors.forEach(actor -> System.out.printf("\t * %s\n", actor));
    }
}
```

① This SQL query will be translated into the same Cypher query of the previous example. The remainder of the method is identical to before.

For more information, see [SQL to Cypher translation](#).

The JDBC specification does not support named parameters, only index-based parameters, starting at 1. So for all `PreparedStatement` instances you need to specify parameters like this:

Listing 4. Using parameters with a `PreparedStatement`

```
var cypher = "CREATE (m:Movie {title: $1})";
try (var con = DriverManager.getConnection(url, username, password);
    PreparedStatement stmt = con.prepareStatement(cypher)) {
    stmt.setString(1, "Test");
    stmt.executeUpdate();
}
```

This is independent of the SQL-to-Cypher translation mechanism:

Listing 5. Using parameters with a `PreparedStatement` (SQL variant)

```
var sql = "INSERT INTO Movie (title) VALUES (?)";
```



```
try (var con = DriverManager.getConnection(url + "?enableSQLTranslation=true",
username, password);
    PreparedStatement stmt = con.prepareStatement(sql)) {
    stmt.setString(1, "Test");
    stmt.executeUpdate();
}
```

To use named parameters, downcast the `PreparedStatement` to `Neo4jPreparedStatement`.

Listing 6. Using named parameters with a `Neo4jPreparedStatement`

```
var match = "MATCH (n:Movie {title: $title}) RETURN n.title AS title";
try (var con = DriverManager.getConnection(url, username, password);
    Neo4jPreparedStatement stmt = con.prepareStatement(match).unwrap
(Neo4jPreparedStatement.class)) {
    stmt.setString("title", "Test");
    try (var resultSet = stmt.executeQuery()) {
        while (resultSet.next()) {
            LOGGER.info(resultSet.getString("title"));
        }
    }
}
```

2.1. Getting a connection via environment variables

If you are happy to depend directly on `org.neo4j.jdbc.Neo4jDriver` and want to get a connection as easy as possible, you might want to use `fromEnv`:

Listing 7. Get a connection from environment variables

```
// import org.neo4j.jdbc.Neo4jDriver;

try (var con = Neo4jDriver.fromEnv().orElseThrow(); ①
    var stmt = con.createStatement();
    var result = stmt.executeQuery(query)) {

    // Same loop as earlier
}
catch (SQLException ex) {
    throw new RuntimeException(ex);
}
```

① Notice how we directly use the concrete driver class here and how the methods return an optional: no connection can be created if the required connection variables are not found.

The `fromEnv` method looks for a few specific system environment variables and it adheres to the [12 factor app](#) principles:

- First, it looks in the system environment

- Second, it looks for a file named `.env` in the current working directory. There are overloads that let you configure the directory and the filename to look for.

The supported variables are:

`NEO4J_URI`

The address or URI of the instance to connect to.

`NEO4J_USERNAME`

(Optional) Username.

`NEO4J_PASSWORD`

(Optional) Password.

`NEO4J_SQL_TRANSLATION_ENABLED`

(Optional) Whether to enable full SQL-to-Cypher translation, defaults to `false`.



Information from both the system environment and the `.env` files are combined. If for example `NEO4J_SQL_TRANSLATION_ENABLED` is in the system environment but not in the `.env` file, it will still be picked up. Given the order of priority, information in the system environment always has precedence over the `.env` file.

This feature is especially useful with Neo4j AuraDB. When creating a new AuraDB instance, you download a `.env` file that you can directly use with the Neo4j JDBC Driver:

Listing 8. Using a `.env` file from AuraDB

```
try (
    var con = Neo4jDriver.fromEnv("Neo4j-cb3d8b2d-Created-2024-02-14.txt")
        .orElseThrow();
    var stmt = con.createStatement();
    var movies = stmt.executeQuery("MATCH (n:Movie) RETURN n.title AS title")
) {
    while (movies.next()) {
        System.out.println(movies.getString("title"));
    }
}
```

3. Distribution

3.1. Driver modules and dependencies

The driver consists of 3 modules:

`neo4j-jdbc`

This is the actual JDBC implementation for Neo4j

neo4j-jdbc-translator-spi

This is the SPI for the default SQL-to-Cypher translation implementation and for any further or custom implementation.

neo4j-jdbc-translator-impl

This is the default and **optional** implementation of an SQL-to-Cypher translator. It provides a somewhat opinionated approach of translating SQL statements into semantically equivalent Cypher statements.



If you just want to use the Neo4j JDBC Driver to run Cypher statements, you only need the module `org.neo4j:neo4j-jdbc`. As long as you are developing in an environment with proper dependency management, this is all you need to declare.

While the translator SPI is a required dependency and gets pulled in via Maven or Gradle, the actual implementation is not. This allows for:

- Opting out of having additional dependencies if you don't need the SQL-to-Cypher translation
- Requiring different JDK baselines or licensing modules for our implementations
- Allowing you to create alternative translators

3.1.1. Dependencies

There are two "dependency-free" modules, which shade everything into one binary artifact each. Those bundles cover a lot of tooling (see [Available bundles](#)).

The driver's direct, compile-time dependencies are listed below:

Listing 9. Dependency tree of the core driver

```
org.neo4j:neo4j-jdbc
+- io.github.cdimascio:dotenv-java
+- io.netty:netty-handler
| +- io.netty:netty-common
| +- io.netty:netty-resolver
| +- io.netty:netty-buffer
| +- io.netty:netty-transport
| +- io.netty:netty-transport-native-unix-common
| \- io.netty:netty-codec
+- org.neo4j:neo4j-cypher-dsl-schema-name-support
+- org.neo4j:neo4j-jdbc-translator-spi
```

3.2. Available bundles

All bundles of the Neo4j JDBC Driver are distributed on Maven Central. The bundles have different characteristics: depending on your use-case or your environment, you can pick one or the other bundle, but not both at once.



All distributions of the Neo4j JDBC driver and all features are compatible with [GraalVM native image](#).

3.2.1. Individual components

Pick this distribution if you have an application that uses proper dependencies management in which you don't have to care about dealing with individual artifacts and their dependencies yourself.



We offer a Maven BOM project, which you should import when using the individual modules, so that you have always a consistent set of versions. The coordinates are `org.neo4j:neo4j-jdbc-bom`.

For Maven, use this dependency declaration:

Listing 10. Maven dependency for the core Neo4j JDBC Driver artifact

```
<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-jdbc</artifactId>
  <version>6.12.3</version>
</dependency>
```

And for Gradle you would want to declare the following runtime dependency:

Listing 11. Neo4j JDBC Driver as Gradle runtime dependency

```
dependencies {
    runtimeOnly 'org.neo4j:neo4j-jdbc:6.12.3'
}
```

If you want to use the SQL-to-Cypher translation from Neo4j, you need to add the following dependency in your Maven build:

Listing 12. Maven dependency for the default SQL to Cypher translator

```
<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-jdbc-translator-impl</artifactId>
  <version>6.12.3</version>
</dependency>
```

and the same coordinates, but for Gradle:

Listing 13. Gradle dependency for the default SQL to Cypher translator

```
dependencies {
    runtimeOnly 'org.neo4j:neo4j-jdbc-translator-impl:6.12.3'
```

```
}
```

3.2.2. Small bundle

Pick this distribution if you work with ETL tools or tooling for relational databases that allow adding the JDBC driver only as single artifacts or that make it unnecessary hard to add additional jars. This bundle does not contain the default SQL-to-Cypher translator! The coordinates of this bundle are `org.neo4j:neo4j-jdbc-bundle` and you can download it from Maven central:

<https://repo.maven.apache.org/maven2/org/neo4j/neo4j-jdbc-bundle/6.12.3/neo4j-jdbc-bundle-6.12.3.jar>



All bundles can be used as normal project dependencies as well. This might be useful for example if your project depends on a different, potentially conflicting Netty version.

3.2.3. Full bundle

This bundle contains the default SQL-to-Cypher translator. Otherwise, it is identical to the [small bundle](#). Its coordinates are `org.neo4j:neo4j-jdbc-full-bundle` and you can download it from Maven central here:

<https://repo.maven.apache.org/maven2/org/neo4j/neo4j-jdbc-full-bundle/6.12.3/neo4j-jdbc-full-bundle-6.12.3.jar>

3.2.4. text2cypher bundle

We also ship a very experimental bundle that we call `text2cypher`:

<https://repo.maven.apache.org/maven2/org/neo4j/neo4j-jdbc-text2cypher-bundle/6.12.3/neo4j-jdbc-text2cypher-bundle-6.12.3.jar>

See [\[^top\]](#) more information.

4. Configuration

4.1. Driver class name

The Neo4j JDBC Driver is `org.neo4j.jdbc.Neo4jDriver`. With modern Java tools, you should not need to touch this class directly, but there are some connection pools and front-ends that will ask you for this. The class is public API.

We also provide `org.neo4j.jdbc.Neo4jDataSource` as `javax.sql.DataSource`.

4.2. URL and connection properties

The canonical URL format for the Neo4j JDBC Driver is

```
jdbc:neo4j://<host>:<port>/<database>?param1=value1&param2=value2
```

The database name and all query parameters are optional and can be omitted. All configuration arguments can be passed either as query parameters or via a `java.util.Properties` object. The latter is sometimes defined by tooling for you.

With regard to authentication, we recommend to follow the JDBC specification, which discourages using any form of URL authentication. All query parameters must be [percent-encoded](#) if they contain special characters, e.g. `...?param1=space%20separated`.



The driver supports the following URI schemes, which tweak the security configuration:

- `neo4j` - No encryption.
- `neo4j+s` - Enables encryption and only accepts SSL certificates from the server that are signed by a known Certificate Authority.
- `neo4j+ssc` - Enables encryption and accepts self-signed certificates (which must be added to the certificate store).

`bolt` URI schemes are not supported.

Last but not least, you can add `http` as sub-protocol when using the Neo4j-JDBC driver against a Neo4j instance from 2025.06 on with the Http Query-API enabled:

```
jdbc:neo4j:http://<host>:<port>/<database>?param1=value1&param2=value2  
jdbc:neo4j:https://<host>:<port>/<database>?param1=value1&param2=value2
```

Usually, Neo4j provides the HTTP port on 7474 or 7473. When you don't specify the port, the Neo4j JDBC Driver defaults to 80 and 443 for http and https respectively.



The query API behaves different in a couple of edge cases. Especially, for big results the driver will materialize them completely and not stream individual records as it does over the binary neo4j protocol. Also, queries that produce `n` results and may fail on the `nth+1` result, will fail immediate when run over HTTP.

Some features such as support for the `Vector` datatype and transactional metadata are not supported at all over HTTP.

The driver accepts the following configuration arguments, either as properties or as URL query parameters:

Table 1. Configuration arguments

Name	Type	Meaning	Default
timeout	Integer	Timeout for connection acquisition in milliseconds	1000
agent	String	User agent	neo4j-jdbc
enableSQLTranslation	Boolean	Flag that enables automatic translation from SQL-to-Cypher (requires a translator on the classpath)	false
cacheSQLTranslations	Boolean	Flag that enables caching of translations. SQL translations are not "free": parsing of SQL costs a bit of time, and so does Cypher rendering. In addition, we might up look up metadata to be able to project individual properties. If this takes too long, translations may be cached.	false
rewritePlaceholders	Boolean	Flag that allows you to use ? as placeholder in Cypher statements (as required by JDBC). These will automatically be rewritten into \$1, \$2 ... \$n, starting at 1, so that the numbering matches the 1-based JDBC index.	true when enableSQLTranslation is false, false otherwise
ssl	Boolean	Optional flag, alternative to neo4j+s. It can be used for example to programmatically enable the full SSL chain.	null

Name	Type	Meaning	Default
sslMode	Enum<SSLMode>	Optional configuration for fine-grained control over SSL configuration. Allowed values are <code>disable</code> , <code>require</code> , <code>verify-full</code> . See <code><ssl_mode, Understanding the SSL mode>></code> .	null
user	String	The username (principal) to use for authentication. NOT RECOMMENDED as URL query parameter for security reasons.	neo4j
password	String	The password (credentials) to use for authentication. NOT RECOMMENDED as URL query parameter for security reasons.	password
authRealm	String	The realm to use for authentication. NOT RECOMMENDED as URL query parameter for security reasons.	null

Name	Type	Meaning	Default
authScheme	String	The authentication scheme to use. NOT RECOMMENDED as URL query parameter for security reasons. Currently supported values are: • basic (default) for basic authentication. • bearer for bearer authentication (SSO). password should be set to the bearer token; user and authRealm have no effect. • kerberos for kerberos authentication. Requires password to be set to the kerberos ticket; user and authRealm have no effect. • none if authentication is disabled on the server. The properties user, password, authRealm have no effect.	basic

Name	Type	Meaning	Default
<code>useBookmarks</code>	<code>boolean</code>	Enables bookmark management for full causal cluster support. This is enabled by default and the recommended setting for all scenarios that use a connection pool. If you disable it, it will only be disabled for the specific connection. Other connections retrieved from the driver instance to the same or to other databases are not affected, and the individual connections will still manage their bookmarks.	<code>true</code>
<code>viewDefinitions</code>	<code>String</code>	A file, http or https URL pointing to a valid JSON file containing definitions for Cypher-backed views .	<code>null</code>
<code>authn.supplier</code>	<code>String</code>	The name of an authentication supplier, can be used to plugin various SSO systems such as Keycloak, with automatic token refresh. See below Custom authentication for more information.	<code>null</code>

Name	Type	Meaning	Default
tryTcpFastOpen	boolean	Setting this value to <code>true</code> makes the driver attempt to use TCP Fast Open. For this to work the following additional requirements must be fulfilled: • The system the driver is running on supports and has TCP Fast Open enabled. • The DBMS to which the driver should connect must also support and enable TCP Fast Open • One of the following additional runtime dependencies (depending on your operating system), must be present on the class- or module path: ◦ netty-transport-native-io_uring (Netty 4.2+ only) ◦ netty-transport-native-epoll ◦ netty-transport-native-kqueue	null



Providing that a valid Netty dependency for transport is added, it will be discovered and used instead of the default NIO transport. Those are operating system specific and valid options are `netty-transport-native-epoll`, `netty-transport-native-kqueue` and on Netty 4.2 or later, `netty-transport-native-io_uring`.

4.3. Getting a driver or a connection instance



This section likely only applies if you use the Neo4j JDBC Driver as part of application development in contrast to using it as part of front-end tool such as [DBeaver](#), [DataGrip](#) or UI-based ETL tools.

The easiest way to acquire a connection is directly through the `java.sql.DriverManager`.

Listing 15. Acquiring a JDBC connection to a Neo4j server

```
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

class Configuration {

    void obtainConnection() throws SQLException {

        var url = "jdbc:neo4j://localhost:7687";
        var username = "neo4j";
        var password = "verysecret";
        var connection = DriverManager.getConnection(url, username, password);
    }

}
```

While our connection implementation is thread-safe, it allows only one concurrent transaction per connection (as dictated by the JDBC specification). For a multi-thread application, use a connection pool. There's [HikariCP](#), but usually application server and containers/frameworks bring their own. It's safe to use any of them, as the Neo4j JDBC Driver does no internal pooling.

If you need access to an instance of the Neo4j driver itself, you can use the following approach:

Listing 16. Acquiring an instance of the Neo4j JDBC Driver

```
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

class Configuration {

    void obtainDriverAndConnection() throws SQLException {

        var url = "jdbc:neo4j://localhost:7687";
        var driver = DriverManager.getDriver(url);

        var properties = new Properties();
        properties.put("username", "neo4j");
        properties.put("password", "verysecret");
        var connection = driver.connect(url, properties);
    }

}
```

```
}  
  
}
```

4.4. Securing your connection by using SSL

The Neo4j JDBC Driver supports the same SSL option of the common Java driver, with the same URL protocols, using `+s` or `+ssc` as additional indicators for the required level of security.

The same configuration can also be achieved with a URL query parameter or an entry in the properties passed to the `DriverManager` or driver instance when asking for a connection. As long as you don't specify contradicting values, it's fine to combine both approaches.

4.4.1. Understanding the SSL mode

The following list is ordered by ascending security:

- `disable` — (default), "I don't care about security and don't want to pay the overhead for encryption."
- `require` — "I want my data to be encrypted, and I accept the overhead. I trust that the network will make sure I always connect to the server I want." (Server must support encryption, no hostname/CA validation is done. This saves the hassle of proper certificates and is only secure on private networks; it should not really be used over public networks.)
- `verify-full` — "I want my data encrypted, and I accept the overhead. I want to be sure that I connect to a server I trust, and that it's the one I specify."



The Neo4j JDBC Driver does not include revocation checks.

The most secure option can also be enabled by setting `ssl=true` either as query parameter or as property entry passed to the `DriverManager`. This option corresponds to `neo4j+s`. On the other hand, `require` corresponds to `neo4j+ssc`.

The additional enum allows us to possibly support additional modes in the future, such as letting the service decide about SSL, or being able to express a preference towards SSL without requiring it.



Neo4j servers can offer both plain Bolt connections and encrypted SSL connection, or just one of them. The fact that you can connect using `neo4j+s` does not mean that you cannot connect using just `neo4j`, or viceversa. This is dependent on the server setup. [Neo4j Aura](#), Neo4j's managed cloud service, only supports encrypted connections, so you *must* use `+s`, `ssl=true`, or `sslMode=verify-full`.

4.4.2. Valid URLs

The following URLs are all valid:

`neo4j+s://xyz.databases.neo4j.io`

Use full verification with the `xzy` instance at AuraDB

`neo4j://xyz.databases.neo4j.io?ssl=true`

The same, but using the shorthand URL parameter

`neo4j://xyz.databases.neo4j.io?sslMode=verify-full`

The same, but using the explicit mode

`neo4j+s://xyz.databases.neo4j.io?ssl=true&sslMode=verify-full`

Not more secure, but does not fail on you

`neo4j+ssc://this.is.a.trustworthy.instance.for.sure.com`

Trust whatever certificate and hostname there is, but do use SSL

`neo4j://my-testing-instance.local`

Use a plain connection.

The driver only refuses contradicting configurations, such as:

- `+s` with `ssl=false`, or `sslMode` set to `disable`
- `+ssc` with `ssl=false`, or any `sslmode` not equal to `require`

Basically, you cannot ask to use SSL and not use it at the same time. The driver offers several mechanism so that you can use a fixed URL with dynamic query parameters, or dynamic URLs, or whatever way of configuring you prefer in a programmatic way.

4.4.3. Using `.dotenv` files

When you create a [Neo4j Aura](#) instance, you will be asked to download a text-file named similar to `Neo4j-9df57663-Created-2023-06-12.txt`. This is essentially a `.dotenv` file containing the information required to connect to the database.

These files can be directly used via `Neo4jDriver.fromEnv()` (see [Getting a connection via environment variables](#)). This method exists in several overloads, which let you configure both filename and directory. Additionally, the builder lets you configure options that are not contained in the files from Aura.

4.5. Causal clustering and bookmarks

The Neo4j JDBC Driver uses bookmarks by default to provide causal consistency in all Neo4j deployments. Bookmarks are managed on the driver level itself, not on the connections spawned by an instance of the driver, so all connections spawned by one instance will partake in the same causal chain of transactions. Connections from different driver instances will not use the same set of bookmarks and there is no built-in machinery that would enable this. If you want or need this, you can directly access the `Neo4jDriver` type to retrieve the current set of known bookmarks and pass them to another driver instance.

4.6. Neo4j transactional metadata

Neo4j supports attaching metadata to transactions, see [SHOW TRANSACTIONS](#). As there is no explicit transaction object in the JDBC specification, the Neo4j JDBC driver needs another mechanism to make these configurable.

The JDBC driver provides the extension interface `Neo4jMetadataWriter`. Our driver, the connection implementation, and all statement variants can be unwrapped accordingly. The configuration is additive: metadata configured for a driver instance will be used for all connections spawned from that driver, connections can add further metadata, and statements can also add their own metadata. Metadata added on a statement has precedence over connection metadata which in turn has precedence over driver metadata:

Listing 17. Configuring transactional metadata

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Map;
import java.util.Properties;
import java.util.logging.Level;
import java.util.logging.Logger;

import org.neo4j.jdbc.Neo4jDriver;
import org.neo4j.jdbc.Neo4jMetadataWriter;

public final class TransactionMetadata {

    private static final Logger LOGGER = Logger.getLogger(TransactionMetadata.class
.getPackageName());

    public static void main(String... args) throws SQLException {
        var url = "jdbc:neo4j://localhost:7687";

        var driver = (Neo4jDriver) DriverManager.getDriver(url);
        driver.withMetadata(Map.of("md_from_driver", "v1", "will_be_overwritten",
"irrelevant"));

        var properties = new Properties();
        properties.put("user", "neo4j");
        properties.put("password", "verysecret");

        try (
            var con = driver.connect(url, properties)
                .unwrap(Neo4jMetadataWriter.class)
                .withMetadata(Map.of("md_from_connection", "v2", "will_be_overwritten
", "xxx")))
            .unwrap(Connection.class);
            var statement = con.createStatement()
```

```

        .unwrap(Neo4jMetadataWriter.class)
        .withMetadata(Map.of("md_from_stmt", "v3", "will_be_overwritten", "v4
    ))
        .unwrap(Statement.class)
    ) {
        try (var result = statement.executeQuery("SHOW TRANSACTIONS YIELD
metadata")) {
            while (result.next()) {
                var metaData = result.getObject("metaData", Map.class);
                LOGGER.log(Level.INFO, "{0}", metaData);
            }
        }
    }
}

```

The output will be similar to this:

```

Juli 17, 2024 1:18:16 PM org.neo4j.jdbc.docs.TransactionMetadata main
INFORMATION: {md_from_driver=v1, md_from_connection=v2, md_from_stmt=v3,
will_be_overwritten=v4}

```

4.7. Logging

4.7.1. Via standard Java settings

To generate a useful and detailed log, you must configure `java.util.logging`. This can be done either through a configuration file or programmatically. The following section includes an example configuration that sets the global logging level to `INFO`, Neo4j JDBC Driver itself to `DEBUG` and everything that is network related to `WARN`:

Listing 18. Logging configuration for the Neo4j JDBC Driver that logs all information

```

.level = INFO
handlers=java.util.logging.ConsoleHandler

# The handler must be configured for the lowest level it should catch
java.util.logging.ConsoleHandler.level=FINEST

# Enables all Logging for the JDBC Driver
org.neo4j.jdbc.level=ALL
# But reduces the noise on the network and the result set
org.neo4j.jdbc.result-set.level=WARNING
org.neo4j.jdbc.network.level=WARNING
# This is an example how to configure the outbound and inbound messages
org.neo4j.jdbc.network.OutboundMessageHandler.level=FINE
org.neo4j.jdbc.network.InboundMessageHandler.level=FINE

```



```
# This produces a date, followed by the level, 3 dashes, the message and the source of
the log if available
java.util.logging.SimpleFormatter.format = %1$tFT%1$tk:%1$tM:%1$tS.%1$tL%1$tz %4$-15s
--- %5$s [%2$s]%n
```

Such a file must be passed to the running Java program as a system property to Java itself with:

```
java -Djava.util.logging.config.file=docs/src/main/resources/logging-to
-console.properties <your program>
```

Capturing logs to a file requires to change the handler:

Listing 19. Logging configuration for the Neo4j JDBC Driver that writes all JDBC logs to a file

```
.level = INFO
handlers=java.util.logging.ConsoleHandler

# Keep everything by default at INFO
java.util.logging.ConsoleHandler.level=INFO

# The handler must be configured for the lowest level it should catch
java.util.logging.FileHandler.level=FINEST
java.util.logging.FileHandler.pattern=jdbc.log
# XML Format by default
# java.util.logging.FileHandler.formatter=java.util.logging.SimpleFormatter

# Change the handler for JDBC
org.neo4j.jdbc.handlers=java.util.logging.FileHandler
# Keep the rest as above
org.neo4j.jdbc.level=ALL
org.neo4j.jdbc.result-set.level=WARNING
org.neo4j.jdbc.network.level=WARNING
org.neo4j.jdbc.network.OutboundMessageHandler.level=FINE
org.neo4j.jdbc.network.InboundMessageHandler.level=FINE

java.util.logging.SimpleFormatter.format = %1$tFT%1$tk:%1$tM:%1$tS.%1$tL%1$tz %4$-15s
--- %5$s [%2$s]%n
```



The diagnosability features of the JDBC driver are based on the standard `java.util.logging` framework. Configuration of that framework itself is out of scope for this documentation. For a complete overview you might want to start at "[Java Logging Overview](#)". The formatter Java uses is the `java.util.Formatter`.

4.7.2. Inside a Spring Boot application

The Neo4j JDBC Driver will integrate itself into the Spring Boot logging configuration and take all relevant format and other settings from there. The levels can be configured with the levels Spring does understand. The following configuration is equivalent to the one above:

Listing 20. Logging configuration for the Neo4j JDBC Driver in a Spring Boot configuration file

```
logging.level.org.neo4j.jdbc=trace
logging.level.org.neo4j.jdbc.result-set=warn
logging.level.org.neo4j.jdbc.network=warn
logging.level.org.neo4j.jdbc.network.OutboundMessageHandler=debug
logging.level.org.neo4j.jdbc.network.InboundMessageHandler=debug
```

4.7.3. Inside a Quarkus application

The Neo4j JDBC Driver will integrate itself into the Quarkus logging configuration and take all relevant format and other settings from there. The levels can be configured with the levels Quarkus does understand. The following configuration is equivalent to the ones above:

Listing 21. Logging configuration for the Neo4j JDBC Driver in a Quarkus configuration file

```
quarkus.log.category."org.neo4j.jdbc".min-level=TRACE
quarkus.log.category."org.neo4j.jdbc".level=TRACE
quarkus.log.category."org.neo4j.jdbc.result-set".level=DEBUG
quarkus.log.category."org.neo4j.jdbc.network".level=WARNING
quarkus.log.category."org.neo4j.jdbc.network.OutboundMessageHandler".level=DEBUG
quarkus.log.category."org.neo4j.jdbc.network.InboundMessageHandler".level=DEBUG
```

4.7.4. SQL Translations

The SQL translators use the logger `org.neo4j.jdbc.translator`; processed SQL and its original sources is logged at level `FINE` on `org.neo4j.jdbc.statement.SQL`.

4.8. Metrics

The Neo4j JDBC Driver will publish metrics via [Micrometer](#). If you put Micrometer on your classpath, no further configuration is necessary. This is for example automatically the case with [Spring Boot Actuator](#) or [Micrometer for Quarkus](#). To make best use of this feature, you will additionally configure your [monitoring system](#).

The Neo4j JDBC Driver does publish the following metrics:

`org.neo4j.jdbc.connections`

a gauge representing the number of open connections, tagged with `url` to which the driver is connected

`org.neo4j.jdbc.statements`

a gauge representing the number of open statements, tagged with both the `url` to which the statement is opened and the JDBC type of the statement (either `Statement`, `PreparedStatement` or `CallableStatement`)

`org.neo4j.jdbc.queries`

a composite meter containing the counts of successful and failed queries and a timer measuring

the duration of queries

`org.neo4j.jdbc.cached-translations`

a gauge representing the number of cached SQL to cypher translations

4.9. Tracing

The Neo4j JDBC Driver supports tracing and will provide traces spans for the following operations on JDBC `Statement` instances:

- `#execute`
- `#executeQuery`
- `#executeUpdate`

and all overloads thereof and in all inheriting types. In addition, when using a JDBC `ResultSet`, it will open a span when the result set is first moved from being before the first row until it is either moved beyond the last row or actively closed. Spans will contain various events, such as when a query has been actually executed by the Neo4j database or when a batch of records has been pulled from the database.

The feature is optional: when you want to use it, you have to bring in additional dependencies. This applies to all distributed versions, including the full bundles, so that an optional feature doesn't make the bundles bigger. For Maven, use this additional dependency declaration:

Listing 22. Maven dependency that allows you to configure tracing

```
<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-jdbc-tracing-micrometer</artifactId>
  <version>6.12.3</version>
</dependency>
```

And for Gradle you would want to declare the following runtime dependency:

Listing 23. Gradle dependency that allows you to configure tracing

```
dependencies {
  runtimeOnly 'org.neo4j:neo4j-jdbc-tracing-micrometer:6.12.3'
}
```

This dependency brings in [Micrometer Tracing](#). Micrometer Tracing allows you to bind to various tracers and exporters, such as Zipkin or OpenTelemetry.

Once you have a Micrometer Tracer, you can configure a `Neo4jDataSource` like this:

Listing 24. A `Neo4jDataSource` spawning connections that trace their usage

```
import javax.sql.DataSource;
```

```

import org.neo4j.jdbc.Neo4jDataSource;
import org.neo4j.jdbc.tracing.micrometer.Neo4jTracingBridge;

import io.micrometer.tracing.Tracer;

public class TracingExampleSetup {

    DataSource neo4jDataSource() {

        Tracer tracer = Tracer.NOOP; ❶
        var neo4jDataSource = new Neo4jDataSource();
        neo4jDataSource.setUrl("jdbc:neo4j://yourhost:7687");
        neo4jDataSource.setPassword("neo4j");
        neo4jDataSource.setUser("verysecret");
        neo4jDataSource.setTracer(Neo4jTracingBridge.to(tracer)); ❷
        return neo4jDataSource;
    }
}

```

- ❶ In most scenarios, your application framework will provide a Micrometer tracer instance for you
- ❷ With this call you enable tracing in the data source and all connections spawned from it

You can also enable tracing on individual connections by unwrapping them into the `Neo4jConnection` extension like this:

Listing 25. Enable tracing for an individual connection

```

import java.sql.Connection;
import java.sql.DriverManager;

import org.neo4j.jdbc.Neo4jConnection;
import org.neo4j.jdbc.tracing.micrometer.Neo4jTracingBridge;

import io.micrometer.tracing.Tracer;

public class TracingExampleSetup {

    Connection connection() {

        Tracer tracer = Tracer.NOOP;
        Neo4jConnection connection = DriverManager.getConnection(
            "jdbc:neo4j://yourhost:7687", "neo4j", "verysecret")
            .unwrap(Neo4jConnection.class); ❶
        connection.setTracer(Neo4jTracingBridge.to(tracer)); ❷
    }
}

```

- ❶ Unwrap the connection into our extension so that you can configure the tracer

② Configure the tracer



In case you neither want to work with a data source nor individual connections, use `Neo4jDriver.registerTracer(Neo4jTracingBridge.to(tracer))` to register the tracer with all Neo4j drivers that are registered with the JDBC `DriverManager`.

4.9.1. Using tracing with Spring Boot

The tracing implementation that is provided by the Neo4j JDBC Driver utilizes Micrometer Tracing, which is well accepted in the Spring Boot ecosystem, but also in Quarkus, Micronaut and others. We provide a few examples, but refer to your frameworks documentation on how to configure tracing.

If you are happy to configure tracing for all connections acquired from the `DriverManager`, configure tracing like this:

Listing 26. Configure tracing for all drivers registered with `DriverManager`

```
import io.micrometer.tracing.Tracer;
import org.neo4j.jdbc.tracing.micrometer.Neo4jTracingBridge;

import org.springframework.beans.factory.InitializingBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class TracingConfiguration {

    @Bean
    InitializingBean neo4jTracingInitializer(Tracer tracer) {
        return () -> Neo4jDriver.registerTracer(Neo4jTracingBridge.to(tracer));
    }
}
```

Otherwise, use a custom datasource as shown below.

Configuring the tracing datasource

Usually, Spring Boot configures a `HikariDataSource` based on your driver. Hikari provides connection pooling, and you should not ditch that. There's no need to create your own data source properties for all configuration values, you can just inject it like this:

Listing 27. Configure a tracing and pooled data source with Spring Boot

```
import javax.sql.DataSource;

import org.neo4j.jdbc.Neo4jDataSource;
import org.neo4j.jdbc.tracing.micrometer.Neo4jTracingBridge;
import org.springframework.boot.autoconfigure.jdbc.DataSourceProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import io.micrometer.tracing.Tracer;

@Configuration
class TracingTestConfiguration {

    @Bean
    DataSource neo4jDataSource(Tracer tracer, DataSourceProperties
dataSourceProperties) {

        var neo4jDataSource = new Neo4jDataSource();
        neo4jDataSource.setUrl(dataSourceProperties.getUrl());
        neo4jDataSource.setPassword(dataSourceProperties.getPassword());
        neo4jDataSource.setUser(dataSourceProperties.getUsername());
        neo4jDataSource.setTracer(Neo4jTracingBridge.to(tracer));

        var cfg = new HikariConfig();
        cfg.setDataSource(neo4jDataSource);
        return new HikariDataSource(cfg);
    }
}

```

Bring `org.springframework.boot:spring-boot-starter-actuator` onto your class path too, and it will configure Micrometer for you.

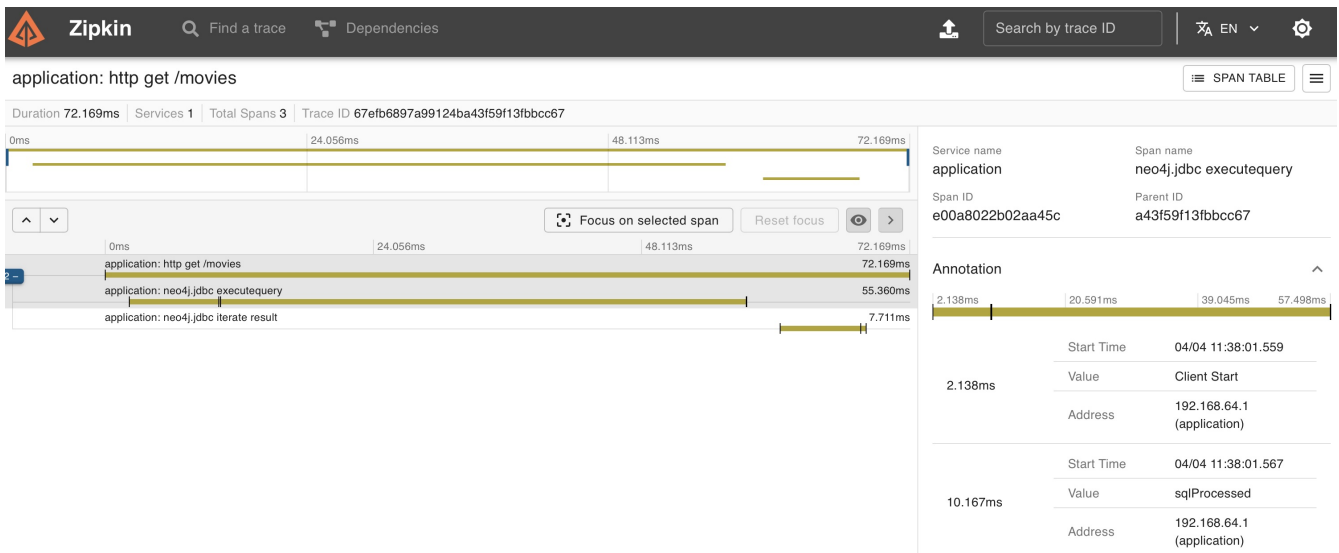


The setup of your data source stays the same: no change is required here for the following examples to work, as our Tracing API will automatically pick the correct tracer through Micrometer!

Exporting to Zipkin

To export your traces to Zipkin, add `io.micrometer:micrometer-tracing-bridge-brave` and `io.zipkin.reporter2:zipkin-reporter-brave` and set `management.zipkin.tracing.export.enabled` to `true` in your Spring application.

You should see spans like this:



Exporting via OpenTelemetry

[OpenTelemetry](#) allows to export not only traces, but metrics and logging too.

Here's an example how to use [Dash0](#) via OpenTelemetry. The exporter should work the same for other services. OpenTelemetry has split its SDK into several modules, hence you will first import their BOM into your build file. Here's an example for Maven:

Listing 28. Snippet from a Maven build file to include the OpenTelemetry BOM

```
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-bom</artifactId>
  <version>1.47.0</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

First you need to bring in the bridge from Micrometer to OpenTelemetry via: `io.micrometer:micrometer-tracing-bridge-otel`, so that Micrometer pushes into that SDK. Next, bring in the following dependencies so that Spring Boot can configure the exporter:

- `io.opentelemetry:opentelemetry-sdk`
- `io.opentelemetry:opentelemetry-exporter-common`
- `io.opentelemetry:opentelemetry-exporter-otlp`
- `io.opentelemetry:opentelemetry-exporter-sender-jdk`

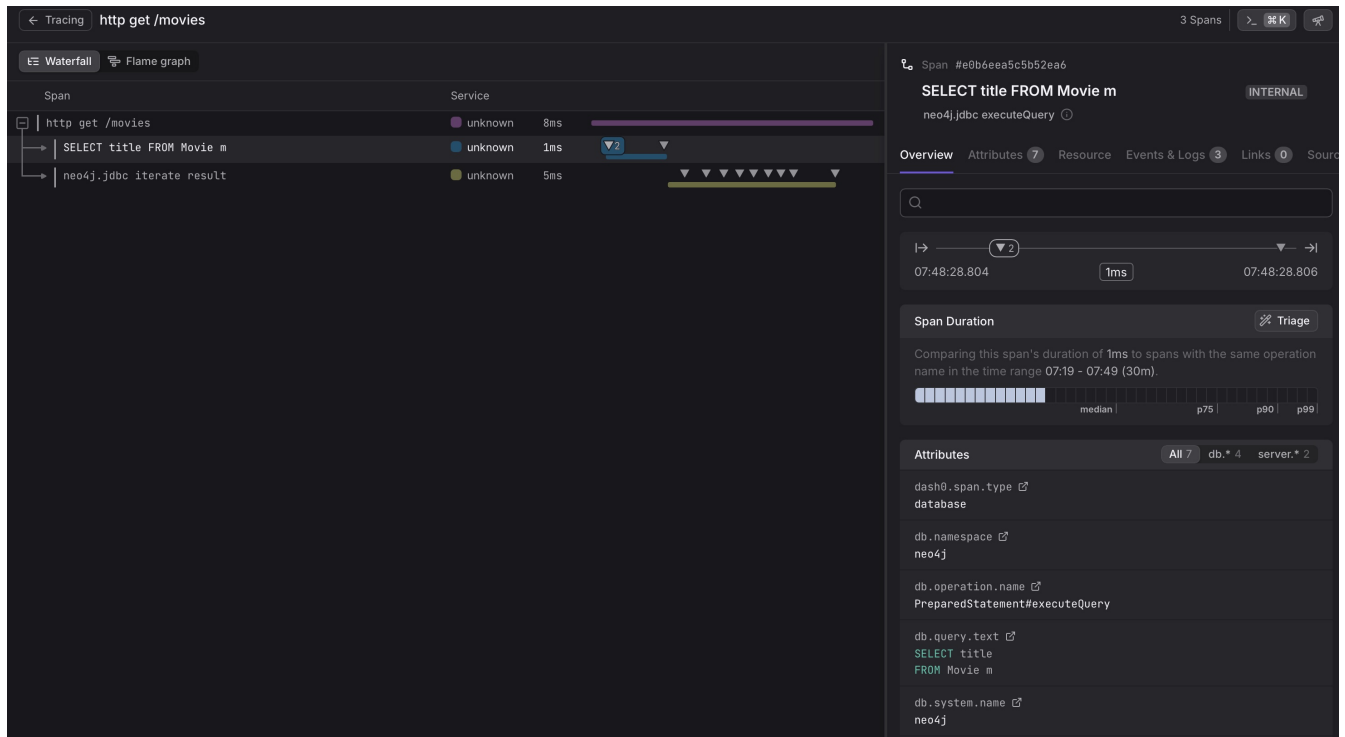
In the case of Dash0, the application configuration will look similar to this:

Listing 29. Using the OpenTelemetry exporter in Spring Boot towards Dash0

```
management.otlp.tracing.export.enabled=true
management.otlp.tracing.endpoint=https://ingress.eu-west-1.aws.dash0.com/v1/traces
management.otlp.tracing.headers.Authorization=Bearer auth_XX_your_token_XX
```

```
management.otlp.metrics.export.step=30s
management.otlp.metrics.export.enabled=true
management.otlp.metrics.export.url=https://ingress.eu-west-1.aws.dash0.com/v1/metrics
management.otlp.metrics.export.headers.Authorization=Bearer auth_XX_your_token_XX
```

The trace from our Movie application looks like this:



4.10. Custom authentication

The Neo4j JDBC driver provides the ability to use a custom authentication supplier in addition to JDBC's standard ways of authentication, such as using `java.sql.DriverManager#getConnection(java.lang.String, java.lang.String, java.lang.String)` or `java.sql.Driver#connect` for authentication with the Neo4j server.

An authentication supplier is in Java terms a `Supplier<org.neo4j.jdbc.authn.api.Authentication>`. An `Authentication` instance is an abstract representation of an arbitrary authentication to be presented to the Neo4j server and can contain a principal and its credentials (username and password), a token or other ways of confirming an identity.

The interface itself offers the following factories:

`Authentication#usernameAndPassword`

Creates a simple username and password based authentication, which is what is used internally already with the default methods

`Authentication#none`

Disables authentication

Authentication#bearer

Creates a bearer-token based authentication

The latter allows to set an expiration date for the token. If the token expires, that is used after the expiration date, the supplier is asked for a new one. This allows configuring the Neo4j JDBC driver in such a way that it can grab tokens from an SSO provider, such as [Keycloak](#) or [Okta](#), and authenticate itself with those tokens against Neo4j and automatically refresh those tokens when they expire.

In addition, there are two interfaces, `CustomAuthentication` and `ExpiringAuthentication` that can be implemented in case the above factories are not sufficient.

There are several ways to pass an authentication supplier:

- Creating a library with your custom authentication supplier and a small factory for it implementing `AuthenticationSupplierFactory`: It will be automatically be loaded into the JDBC driver and be used when the JDBC property `authn.supplier` is set to the name of the factory
- On the driver itself, when unwrapped to a `Neo4jDriver`, there is `driver#connect(String, Properties, Supplier)`. The given supplier will have precedence over any authentication information in the `properties` parameter
- Setting a supplier instance on a particular instance of the driver with `driver#setAuthenticationSupplier(Supplier)`
- Registering a global supplier instance with `Neo4jDriver#registerAuthenticationSupplier` that will be used for all connections spawned from the `DriverManager`
- The convenient builder methods present as static methods on `Neo4jDriver` allowing to configure the driver from the environment (see [Getting a connection via environment variables](#)) allow to specify the supplier, too

The precedence is as follows:

1. If there's a supplier passed to `connect` it has precedence over the drivers locally configured supplier respectively the global one and any properties
2. If there's a property named `authn.supplier` in the JDBC properties and an authentication supplier factory with that name is available, a supplier will be sourced from that factory and used
3. The supplier configured per driver or the global one has precedence over any properties
4. Properties will be used as per JDBC standard when there is no explicit or implicit authentication supplier

The driver will only call the supplier when no token has been retrieved yet or when the first token expires. Any authentication supplier implementation can safely assume that the second call for a token means the first token is expired, so that the supplier does not need to manage state.



The underlying bolt protocol and the Neo4j Server might not work reliable with very short-lived authentication tokens (i.e. with a lifespan under 5 seconds). This restriction might be lifted in the future, but as of release 6.6.0 you might want to

refrain from very short-lived tokens.

4.10.1. Keycloak example

The following classes shows an example based on Keycloaks `AuthzClient`. The supplier will grab tokens from a Keycloak instance and store their expiration date along with the JWT token. The initial ask for a token will always be answered with a completely new one, each follow-up call will use the client to refresh the token.



There is no need to copy the below code into your own application. We actually do ship this authentication supplier with the optional `org.neo4j:neo4j-jdbc-authn-kc` module. The supplier is part of our semver api contract and can either be used directly or via the service loader mechanism, which is also demonstrated in the following documentation.

Listing 30. `KCAuthenticationSupplier`, as an example of an authentication supplier refreshing a token when necessary

```
import java.io.IOException;
import java.io.UncheckedIOException;
import java.time.Instant;
import java.util.Base64;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicReference;
import java.util.function.Supplier;

import com.fasterxml.jackson.jr.ob.JSON;
import org.keycloak.authorization.client.AuthzClient;
import org.keycloak.authorization.client.Configuration;
import org.keycloak.authorization.client.util.Http;
import org.keycloak.representations.AccessTokenResponse;
import org.neo4j.jdbc.authn.spi.Authentication;

public final class KCAuthenticationSupplier implements Supplier<Authentication> {

    public static Supplier<Authentication> of(String user, String password,
        Configuration configuration) {
        return new KCAuthenticationSupplier(user, password, configuration);
    }

    private final String username;

    private final String password;

    private final Configuration cfg;

    private final AuthzClient authzClient;

    private final Http http;
```

```

private final String url;

private final AtomicReference<TokensAndExpirationTime> currentToken = new
AtomicReference<>();

KCAuthenticationSupplier(String user, String password, Configuration cfg) {
    this.username = user;
    this.password = password;
    this.cfg = cfg;
    this.authzClient = AuthzClient.create(cfg);
    this.url = "%s/realms/%s/protocol/openid-connect/token".formatted(cfg
.getAuthServerUrl(), cfg.getRealm());
    this.http = new Http(cfg, cfg.getClientCredentialsProvider());
}

@Override
public Authentication get() {
    return this.currentToken.updateAndGet(previous -> {
        if (previous == null) {
            return get0();
        }
        return refresh0(previous.refreshToken());
    }).toAuthentication();
}

TokensAndExpirationTime get0() {
    try {
        return TokensAndExpirationTime.of(this.authzClient.obtainAccessToken(this
.username, this.password));
    }
    catch (IOException ex) {
        throw new UncheckedIOException(ex);
    }
}

TokensAndExpirationTime refresh0(String refreshToken) {
    try {
        return TokensAndExpirationTime.of(this.http.<AccessTokenResponse>post(
this.url)
            .authentication()
            .client()
            .form()
            .param("grant_type", "refresh_token")
            .param("refresh_token", refreshToken)
            .param("client_id", this.cfg.getResource())
            .param("client_secret", (String) this.cfg.getCredentials().get("
secret"))
            .response()
            .json(AccessTokenResponse.class)
            .execute());
    }
}

```

```

        catch (IOException ex) {
            throw new UncheckedIOException(ex);
        }
    }

    record TokensAndExpirationTime(String accessToken, Instant expiresAt, String
refreshToken) {

        static final Base64.Decoder DECODER = Base64.getDecoder();

        static TokensAndExpirationTime of(AccessTokenResponse accessTokenResponse)
throws IOException {
            var chunks = accessTokenResponse.getToken().split("\\.");
            var payload = JSON.std.mapFrom(DECODER.decode(chunks[1]));
            var exp = Instant.ofEpochSecond(((Number) payload.get("exp")).longValue()
);
            return new TokensAndExpirationTime(accessTokenResponse.getToken(), exp,
accessTokenResponse.getRefreshToken());
        }

        Authentication toAuthentication() {
            return Authentication.bearer(this.accessToken, this.expiresAt);
        }
    }
}

```

Again, for using the above supplier add the following dependency to your class- or module-path and don't copy the code, which is only displayed here as an example how you would write your own authentication supplier:

Listing 31. Maven dependency for the core Neo4j JDBC Driver artifact

```

<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-jdbc-authn-kc</artifactId>
  <version>6.12.3</version>
</dependency>

```

The easiest way to use the supplier now is via the build-in loader, so that you can stick purely to the JDBC api:

Listing 32. Using a custom authentication supplier via the `authn.supplier` config option:

```

@Test
void authenticationSupplierFactoriesShouldWork() throws SQLException {

    var properties = new Properties();
    properties.put("enableSQLTranslation", "true");
}

```

```

properties.put("user", "william.foster");
properties.put("password", "d-fens");

properties.put("authn.supplier", "kc"); ①
properties.put("authn.kc.authServerUrl", "http://localhost:%d".formatted(KEYCLOAK
.getHttpPort())); ②
properties.put("authn.kc.realm", "neo4j-sso-test");
properties.put("authn.kc.clientId", "neo4j-jdbc-driver");
properties.put("authn.kc.clientSecret", "QcWXnTg8qJpVMnIvm8Ev8gp1PqJitZu4");

try (var connection = DriverManager.getConnection(neo4jUri, properties); ③
    var stmt = connection.createStatement();
    var rs = stmt.executeQuery("SELECT 1")) {
    assertThat(rs.next()).isTrue();
    assertThat(rs.getInt(1)).isOne();
}
}

```

- ① Indicate that you want to use the Keycloak authentication supplier (named `kc` in our distribution)
- ② This and the following properties will be passed to the authentication system so that it can create authentication with Keycloak
- ③ This is a pure JDBC spec call

In case you don't want to use the automatic loader for authentication supplier factories, or you did write a completely custom authentication supplier and don't want to provide `services` and corresponding module descriptions, there are several other ways of using an authentication supplier.

The most JDBC complaint way is registering it for the global driver like this:

Listing 33. Registering an instance of the authentication supplier globally

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.util.function.Supplier;

import org.keycloak.authorization.client.Configuration;
import org.neo4j.jdbc.authn.kc.KCAuthenticationSupplier;
import org.neo4j.jdbc.authn.spi.Authentication;
import org.neo4j.jdbc.Neo4jDriver;

class Example {

    public static void main(String...a) throws Exception {
        Supplier<Authentication> authenticationSupplier = KCAuthenticationSupplier.of(
("user", "pass", new Configuration());
        Neo4jDriver.registerAuthenticationSupplier(authenticationSupplier);
        Connection connection = DriverManager.getConnection(

```

```
"neo4j+s://xyz.databases.neo4j.io");
    }
}
```

The advantage here is that it makes sure that all applications that use the Neo4j JDBC Driver in a JDBC compliant way over the `DriverManager` will automatically use the authentication supplier. Hence, the above setting is most likely the easiest way for integrations with frameworks like Spring Boot, in which you could configure the global supplier through an `org.springframework.beans.factory.InitializingBean`. Alternatively use the driver builder or manually create an instance of the driver:

Listing 34. Creating an explicit instance of `Neo4jDriver` so that the `connect` overload taking in an authentication supplier is visible

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Properties;
import java.util.function.Supplier;

import org.keycloak.authorization.client.Configuration;
import org.neo4j.jdbc.authn.kc.KCAuthenticationSupplier;
import org.neo4j.jdbc.authn.spi.Authentication;
import org.neo4j.jdbc.Neo4jDriver;

class Example {

    public static void main(String...a) throws Exception {
        Supplier<Authentication> authenticationSupplier = KCAuthenticationSupplier.of(
("user", "pass", new Configuration());
        Neo4jDriver.registerAuthenticationSupplier(authenticationSupplier);
        Neo4jDriver driver = new Neo4jDriver();
        Connection connection = driver.connect("neo4j+s://xyz.databases.neo4j.io", new
Properties(), authenticationSupplier);
    }
}
```

This alternative is feasible in any application for which you are in sole control of creating connections.



If you need to against Keycloak, the Neo4j JDBC driver does ship everything you need, and you are good to go. Just include the Maven module and apply the recommended way of configuring the supplier via JDBC properties. Write a custom authentication supplier only for other authentication needs.

5. Metadata

5.1. Catalog and schema

All the methods on connection level dealing with metadata and information about the available content provide information in terms that are defined in the SQL standard, including catalogs and schemas.

In most relational databases, a catalog is equivalent to a specific database on a server or cluster, and the schema refers to the collection of tables, views and procedures in that catalog.

From the SQL 1992 standard (find an archived copy [here](#)):

(4.12) Catalogs are named collections of schemas in an SQL-environment. An SQL-environment contains zero or more catalogs. A catalog contains one or more schemas, but always contains a schema named INFORMATION_SCHEMA that contains the views and domains of the Information Schema.

This driver only supports a single catalog, which is equal to the Neo4j database to which the driver is connected to. Filtering on a catalog pattern is not supported in metadata queries, all values except literal `null`, the empty or blank string or the name of the current database will lead to an exception.

The same standard defines schemas as follows:

(4.11) An SQL-schema is a persistent descriptor that includes:

[...] the descriptor of every component of the SQL-schema.

In this International Standard, the term "schema" is used only in the sense of SQL-schema. Each component descriptor is either a domain descriptor, a base table descriptor, a view descriptor, an assertion descriptor, a privilege descriptor, a character set descriptor, a collation descriptor, or a translation descriptor. The persistent objects described by the descriptors are said to be owned by or to have been created by the <authorization identifier> of the schema.

We report the literal name `public` as schema for any result component of a metadata result set. We support querying for objects in any schema, however only literal `null` or `public` will potentially produce a result.

Labels will be reported as table objects with the `TABLE_TYPE` being literal `TABLE`.

5.1.1. Summary

- Catalog: Always equals to the current database; filtering on anything else except an empty string will error.

- Schema: Always `public`; filtering on `public` and literal `null` will yield result, anything else won't.
- Table descriptors: Reported as `TABLE` in the `TABLE_TYPE` column.

5.2. Labels to tables

The driver computes table names from node types as follows:

- Single label nodes will map naturally to a table name, the single label will become the table name
 - The label name will be taken as is and will be case-sensitive. The labels `Movie`, `movie`, `MOVIE` will result in three tables in the metadata
 - This is in line with the default SQL-to-Cypher translation
- Node type combinations map to individual table names, so that a node with the labels `Action&Movie` will appear in two tables, which is in line with the current default SQL to Cypher translation

5.3. Primary keys

The driver uses available constraint information and will figure out if there is a single, unique constraint on a label. If that's the case, the constrained property will be assumed to be the primary key. This will also work for unique constraints over multiple properties, in SQL lingo composite primary keys. If there is no unique constraint or more than one, we assume the `v$id` virtual columns for the `elementId` value to be primary keys.

6. SQL to Cypher translation

6.1. Introduction

The translation of queries from SQL to Cypher is an optional feature of this driver and it consists of two parts:

- The translator SPI, located in the module `org.neo4j:neo4j-jdbc-translator-spi`. It consists of two interfaces: `SqlTranslatorFactory` and the actual `SqlTranslator`.
- A concrete implementation of this SPI, published as `org.neo4j:neo4j-jdbc-translator-impl`.

The latter is covered in "[Using the default translator](#)" and available in the "full bundle", described in [Available bundles](#). The former is provided for two reasons: it allows us to distribute the driver with and without the bundled, default translator and allows you to run your custom translator.

Translators can be chained, and there can be as many translators on the classpath as you want. Their precedence is configurable, with our default implementation having the lowest precedence. Thus, you can for example have a custom translator that takes care of a fixed set of queries and, if it receives a query it cannot translate, it will pass it down to our implementation.

Translating arbitrary SQL queries to Cypher is an opinionated task, as there is no right way to map

table names to graph objects: a table name can be used as-is as a label, or may be transformed into a singular form, etc. Mapping relationships is even trickier: should relationship types derive from a join table, a join column (in that case, which one?), or a foreign key?

We believe our assumptions are appropriate for various use cases and instead of providing configuration to cater for all scenarios, we offer the possibility to write your own translation layer. The driver will use the standard Java service loader mechanism to find an implementation of the SPI on the module- or classpath.



Some tools (like Tableau) use a class-loader that won't let the driver use the standard Java service loader mechanism. For these scenarios, we provide an additional configuration property named `translatorFactory`. Set this to `DEFAULT` to directly load our default implementation or to a fully-qualified classname for any other factory. **Be aware** that either our default implementation or your custom one must be on the classpath.

6.2. Translating SQL to Cypher

There's only one requirement to enable the SQL-to-Cypher translation: you have to have one module implementing the SPI on the classpath. This is **automatically** the case if you use the full-bundle (`org.neo4j:neo4j-jdbc-full-bundle`). In that case, you **don't** need to add any other dependency. If you use the individual distribution or the "small" bundle `org.neo4j:neo4j-jdbc-bundle`, you must add the artifact `org.neo4j:neo4j-jdbc-translator-impl`.

The implementation will be automatically loaded. If you use the translation on a case-by-case basis, it will be lazily loaded (i.e no additional classes are touched or loaded into memory). If you configure automatic translation for all statements, the implementation will be eagerly loaded. There are no further configuration options with regard to loading the implementation.

6.2.1. On a case-by-case basis

The translator can be used on a case-by-case basis through the official JDBC API `nativeSQL`, which you find on the `java.sql.Connection` class. With the following imports:

```
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;
import java.util.logging.Logger;
```

You just pass your SQL statement to `nativeSQL` and you will get Cypher back:

```
try (var connection = DriverManager.getConnection(url, username, password)) {
    var sql = connection.nativeSQL("SELECT * FROM Movie n");
    assert """
        MATCH (n:Movie)
        RETURN *""".equals(sql);
}
```

```
}
```

6.2.2. For all queries

If you open the connection to your Neo4j instance using `enableSQLTranslation=true` either as URL parameter or configuration property, all statements will be translated from SQL to Cypher. If you configure the driver in this way, the translator will be eagerly loaded.

```
var url = "jdbc:neo4j://localhost:7687?enableSQLTranslation=true";
try (var connection = DriverManager.getConnection(url, username, password);
    var stmt = connection.createStatement();
    var result = stmt.executeQuery("SELECT n.title FROM Movie n")) {
    while (result.next()) {
        LOGGER.info(result.getString("n.title"));
    }
}
```

Sometimes you may need to fall back to Cypher for some statements, either to use constructs that you cannot express with SQL, or because our default translator cannot handle your query. We offer a special comment that you can use as a hint in your statement to stop automatic translation: `/*+ NEO4J FORCE_CYPHER */`.

```
var url = "jdbc:neo4j://localhost:7687?enableSQLTranslation=true";
var query = """
    /*+ NEO4J FORCE_CYPHER */
    MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
        ((:Stop)-[:NEXT]->(:Stop)){1,3}
        (a:Stop)-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
    RETURN localtime(d.departs) AS departureTime,
        localtime(a.arrives) AS arrivalTime
    """;
try (var connection = DriverManager.getConnection(url, username, password);
    var stmt = connection.createStatement();
    var result = stmt.executeQuery(query)) {
    while (result.next()) {
        LOGGER.info(result.getTime("departureTime").toString());
    }
}
```

6.2.3. Possible error scenarios

A `NoSuchElementException` with a message of `No SQL translators available` will be thrown when there is no implementation of the SQL to Cypher translator available, and you either used `java.sql.Connection.nativeSQL` or enabled automatic translation. The exception will be thrown when you access the method or eagerly on opening a connection in the latter case.

6.3. Using the default translator

6.3.1. Supported SQL dialects

Our default translator uses the OSS parser from [jOOQ](#), which supports a broad spectrum of SQL dialects already. We picked the generic, default dialect of jOOQ as our default dialect, but you can overwrite this in the SQL to Cypher configuration using the parameter `s2c.sqlDialect` with one of the supported dialects listed in [configuration below](#). `POSTGRES` can be a good choice for several integrations.

Bear in mind though that any shortcomings in the translation are probably not due to a lack in the parser, but due to the lack of an obvious, semantically equivalent Cypher construct. That means we might be able to parse a certain piece of SQL, but are unable to translate it into something meaningful that Neo4j can understand without additional, contextual information.

6.3.2. Configuration

The default implementation provides a number of configuration settings. **They must be prefixed with `s2c` in the URL or config options:**

Name	Meaning	Default
<code>parseNameCase</code>	Whether to parse table names as is or not.	<code>true</code>
<code>tableToLabelMappings</code>	A map from table names to labels.	An empty map
<code>joinColumnsToTypeMappings</code>	A map from column names to relationship types.	An empty map
<code>prettyPrint</code>	Whether to format the generated Cypher or not.	<code>true</code>
<code>alwaysEscapeNames</code>	Whether to always escape names.	Unless explicitly configured <code>false</code> when pretty printing is on, otherwise <code>true</code> .
<code>sqlDialect</code>	Which dialect to use when parsing. Supported values are <code>POSTGRES</code> , <code>SQLITE</code> , <code>MYSQL</code> , <code>H2</code> , <code>HSQldb</code> , <code>DERBY</code> and <code>DEFAULT</code> .	<code>DEFAULT</code>
<code>relationshipPattern</code>	The pattern that is used for automatically inferring a relationship from a table-name. ¹	A pattern that matches <code>ALabel_RELATIONSHIP_TYPE_AnotherLabel</code> (i.e. relationship type must be upper-case and can contain underscores)



¹ The pattern must be a valid regular expression with three capturing groups: First group will be interpreted as start or left-hand-side node, second group will be interpreted as the relationship type and the third as end or right-hand-side node.

Named groups `start`, `reltype` and `end` are supported too. `null` or a blank pattern disables automatic inference.

The next few examples use the `properties` config to avoid terrible long URLs in this documentation, but all the attributes can be specified via URL as well.

Listing 35. Disable pretty printing; only escape if necessary; configure dedicated table mappings

```
var properties = new Properties();
properties.put("username", "neo4j");
properties.put("password", "verysecret");
properties.put("enableSQLTranslation", "true");
properties.put("s2c.prettyPrint", "false");
properties.put("s2c.alwaysEscapeNames", "false");
properties.put("s2c.tableToLabelMappings",
    "people:Person;movies:Movie;movie_actors:ACTED_IN");

var url = "jdbc:neo4j://localhost:7687";
var query = """
    SELECT p.name, m.title
    FROM people p
    JOIN movie_actors r ON r.person_id = p.id
    JOIN movies m ON m.id = r.person_id""";
try (var connection = DriverManager.getConnection(url, properties)) {
    var sql = connection.nativeSQL(query);
    assert "MATCH (p:Person)-[r:ACTED_IN]->(m:Movie) RETURN p.name, m.title".equals
(sql);
}
```

Listing 36. Parse table names into upper case

```
var properties = new Properties();
properties.put("username", "neo4j");
properties.put("password", "verysecret");
properties.put("enableSQLTranslation", "true");
properties.put("s2c.parseNameCase", "UPPER");

var url = "jdbc:neo4j://localhost:7687";
var query = "SELECT * FROM people";
try (var connection = DriverManager.getConnection(url, properties)) {
    var sql = connection.nativeSQL(query);
    assert """
        MATCH (people:PEOPLE)
        RETURN *""".equals(sql);
}
```

Named parameter syntax in the SQL parser defaults to `:name` (such as supported by Oracle, JPA, Spring, a colon followed by a name). The following example changes that prefix to `$` (the same prefix that Cypher uses):

Listing 37. Change parameters prefix and add mappings for join columns

```
var properties = new Properties();
properties.put("username", "neo4j");
properties.put("password", "verysecret");
properties.put("enableSQLTranslation", "true");
properties.put("s2c.parseNamedParamPrefix", "$");
properties.put("s2c.joinColumnsToTypeMappings", "people.movie_id:DIRECTED");

var url = "jdbc:neo4j://localhost:7687";
var query = """
    SELECT *
    FROM people p
    JOIN movies m ON m.id = p.movie_id
    WHERE p.name = $1
    """;
try (var connection = DriverManager.getConnection(url, properties)) {
    var sql = connection.nativeSQL(query);
    assert """
        MATCH (p:people)-[:DIRECTED]->(m:movies)
        WHERE p.name = $1
        RETURN *""".equals(sql);
}
```

This is helpful when a tool generates names like that and does not allow customization.

6.3.3. Supported statements

The following statements are all under tests and describe what you can expect from the default translation layer:

6.3.4. Translation concepts

Table names to labels

The most simple **SELECT** statement to translate is one without **FROM** clause, such as:

```
SELECT 1
```

It is equivalent to a Cypher **RETURN**:

```
RETURN 1
```

SELECT statements without **JOIN** clauses are pretty straightforward to translate. The challenge here is how to map the table name to labels:

- We parse the SQL statement case-sensitive by default

- Table names are mapped to node labels
- Table aliases are used as identifiable symbolic names

```
SELECT t.a, t.b
FROM My_Table ①
  AS t ②
WHERE t.a = 1
```

① Will be used as the label to match, as-is, i.e. `My_Table`

② The table alias will become the node alias

The whole construct will be translated to

```
MATCH (t:My_Table)
WHERE t.a = 1
RETURN t.a, t.b
```

Table aliases are optional, if you omit them, we derive aliases from the labels and types. If you inspect the translated queries, we recommend using aliases, as this makes the queries better readable.

Star-Selects

A star- or *-select comes in different forms:

Unqualified

```
SELECT * FROM table
```

Qualified

```
SELECT t.* FROM table t
```

And a variant, selecting the relation itself: `SELECT t FROM table t`.

We make use of this to let you decide whether you want to return Neo4j nodes and relationships as entities, maps, or flattened to individual columns. The latter requires our translator to have access to the schema of the underlying Neo4j database. The following sections describe the use-cases.

Projecting individual properties

Don't use a star-select but enumerate the properties:

```
SELECT m.title FROM Movie m
```

The table alias will be used as a symbolic name:

```
MATCH (m:Movie)
```

```
RETURN m.title;
```

You can omit the table alias:

```
SELECT title FROM Movie
```

The lower-cased table name will be the symbolic name, and when it's only one label to be matched, aliased so that it can be easily retrieved by that name.

```
MATCH (movie:Movie)
RETURN movie.title AS title;
```

Accessing JDBC columns by name leads to code that is hard to maintain, as column renamings impact code as well. To avoid this, alias the column:

```
SELECT title AS title FROM Movie
```

So that it has a stable, well-known name:

```
MATCH (movie:Movie)
RETURN movie.title AS title;
```

Projecting all properties

A **SELECT *** statement gets translated differently depending on whether the connection to the Neo4j database is available.

```
SELECT * FROM Person p
```

If you are offline, you will get the following Cypher statement:

```
MATCH (p:Person) RETURN *
```

The above query will return one column (**p**) containing a Neo4j node. This is usually not what you expect in the relational world. If you run the translation online and Neo4j metadata can be retrieved, you will get a statement that flattens the properties of each node and relationship, plus their element IDs:

In case the **Person** node has properties **born** and **name**,

```
SELECT * FROM Person p
```

you will get this Cypher statement

```
MATCH (p:Person)
RETURN elementId(p) AS 'v$id',
       p.born AS born, p.name AS name
```

This works well with multiple tables as well (Movie has properties `title` and `released`):

```
SELECT * FROM Person p JOIN Movie m ON m.id = p.acted_in
```

```
MATCH (p:Person)-[acted_in:ACTED_IN]->(m:Movie)
RETURN elementId(p) AS 'v$id', p.born AS born, p.name AS name,
       elementId(m) AS 'v$id1', m.title AS title, m.released AS released
```

We append increasing numbers to column names to clashing ones (ex. with `name` and `remark` properties both in `Movie` and `Person`):

```
SELECT * FROM Person p JOIN Movie m ON m.id = p.acted_in
```

Note the increasing numbers per duplicate name:

```
MATCH (p:Person)-[acted_in:ACTED_IN]->(m:Movie)
RETURN elementId(p) AS 'v$id',
       p.born AS born, p.name AS name, p.remark AS remark,
       elementId(m) AS 'v$id1',
       m.name AS name1, m.released AS released, m.remark AS remark1
```

The following example uses a join-table to access relationships (we explain this later in this manual [when discussing joins](#)), and the flattening of properties works here as well:

```
SELECT *
FROM people p
JOIN movie_actors r ON r.person_id = p.id
JOIN movies m ON m.id = r.person_id
```

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
RETURN elementId(p) AS 'v$id',
       p.born AS born, p.name AS name,
```



```
elementId(p) AS `v$person_id`,  
elementId(r) AS `v$id1`, r.role AS role,  
elementId(m) AS `v$movie_id`,  
elementId(m) AS `v$id2`,  
m.title AS title, m.released AS released
```

Listing 38. Ordering without specifying a table alias

```
SELECT * FROM Person p ORDER BY name ASC
```

```
MATCH (p:Person)  
RETURN elementId(p) AS `v$id`,  
       p.born AS born, p.name AS name  
ORDER BY p.name
```

A qualified alias can be used as well. If no Neo4j metadata is available, you will get a map of properties of the node/relationship:

```
SELECT m.*, p.*  
FROM Person p  
JOIN Movie m ON m.id = p.acted_in
```

The corresponding columns must be downcast to a map in JDBC:

```
MATCH (p:Person)-[acted_in:ACTED_IN]->(m:Movie)  
RETURN m{.*} AS m, p{.*} AS p
```

If we add more data (ex. **born** and **name** to **Person**), the qualified star will project all of them (note how we also project one single, known column from the **Movie** table):

```
SELECT p.*, m.title AS title  
FROM Person p  
JOIN Movie m ON m.id = p.acted_in
```

```
MATCH (p:Person)-[acted_in:ACTED_IN]->(m:Movie)  
RETURN elementId(p) AS `v$id`, p.born AS born, p.name AS name, m.title AS title
```

Returning nodes and relationships

A statement that projects a table alias such as

```
SELECT m FROM Movie m
```

will result in a Cypher statement returning the matched node as node.

```
MATCH (m:Movie)  
RETURN m;
```

A node can be aliased as well:

```
SELECT m AS node FROM Movie m
```

```
MATCH (m:Movie)  
RETURN m AS node;
```

Un-aliased tables can be used as well:

```
SELECT movie FROM Movie
```

```
MATCH (movie:Movie)  
RETURN movie;
```

Multiple entities are supported, too:

```
SELECT p, r, m FROM Person p  
JOIN ACTED_IN r ON r.person_id = p.id  
JOIN Movie m ON m.id = r.movie_id
```

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie) RETURN p, r, m
```

Comparing SQL with Cypher examples

The source of the following examples is: [Comparing SQL with Cypher](#).

Find all Products

Select and Return Records

Select everything from the `products` table.

```
SELECT p.*  
FROM products as p
```

Similarly, in Cypher, you just **MATCH** a simple pattern: all nodes with the **label** `Product` and **RETURN** them.

```
MATCH (p:Product)  
RETURN p{.*} AS p
```

The above query will project all properties of the matched node. If you want to return the node itself, select it without using the asterisk:

```
SELECT p  
FROM products as p
```

```
MATCH (p:Product)  
RETURN p
```

Field Access, Ordering and Paging

It is more efficient to return only a subset of attributes, like `ProductName` and `UnitPrice`. And while we are at it, let's also order by price and only return the 10 most expensive items. (Remember that labels, relationship-types and property-names are **case sensitive** in Neo4j.)

```
SELECT p.`productName`, p.`unitPrice`  
FROM products as p  
ORDER BY p.`unitPrice` DESC  
LIMIT 10
```

```
MATCH (p:Product)  
RETURN p.productName, p.unitPrice ORDER BY p.unitPrice DESC LIMIT 10
```

In the same fashion, we do support **OFFSET** queries:

SQL	Cypher equivalent
SELECT name, age FROM People p ORDER BY name LIMIT 5 OFFSET 10	MATCH (p:People) RETURN p.name AS name, p.age AS age ORDER BY p.name SKIP 10 LIMIT 5
SELECT name FROM People p ORDER BY name LIMIT 3 OFFSET 0	MATCH (p:People) RETURN p.name AS name ORDER BY p.name SKIP 0 LIMIT 3
SELECT department, count(*) AS cnt FROM People p GROUP BY department ORDER BY department LIMIT 2 OFFSET 1	MATCH (p:People) RETURN p.department AS department, count(*) AS cnt ORDER BY p.department SKIP 1 LIMIT 2

The default order direction will be translated as is:

```
SELECT * FROM Movies m ORDER BY m.title
```

```
MATCH (m:Movies)
RETURN * ORDER BY m.title
```

DISTINCT projections

The **DISTINCT** keyword for projections is handled:

```
SELECT DISTINCT m.released FROM Movies m
```

```
MATCH (m:Movies)
RETURN DISTINCT m.released
```

It works with ***** projections as well:

```
SELECT DISTINCT m.* FROM Movies m
```

```
MATCH (m:Movies)
RETURN DISTINCT m {.*} AS m
```

However, as the qualified asterisks will use metadata if available, the translation with a database connection is different:

```
SELECT DISTINCT m.* FROM Movies m
```

```
MATCH (m:Movies)
RETURN DISTINCT elementId(m) AS 'v$id', m.title AS title, m.released AS released
```

Note that each row includes the Neo4j element ID, making each row unique. This being said, the **DISTINCT** clause is of limited use with the asterisk.

6.3.5. Expressions

Most SQL expressions have corresponding Cypher expressions and can be translated straightforward.

Literal Values

Literal values are 1:1 translations.

```
SELECT  
  1, TRUE, FALSE, NULL, 'a'
```

```
RETURN 1, TRUE, FALSE, NULL, 'a'
```

Arithmetic expressions

Arithmetic expressions are 1:1 translations.

```
SELECT  
  1 + 2,  
  1 - 2,  
  1 * 2,  
  1 / 2,  
  square(2)
```

Note that the underlying tech of the default translator uses [Cypher-DSL](#) internally, which will wrap arithmetic (and logical) expressions with parentheses:

```
RETURN  
  (1 + 2),  
  (1 - 2),  
  (1 * 2),  
  (1 / 2),  
  (2 * 2)
```

Functions

Numeric functions

We can translate all numeric functions that are supported by Neo4j's Cypher implementation: [Mathematical functions - Numeric](#):

SELECT

```
abs(1),  
ceil(1),  
floor(1),  
round(1),  
round(1, 1),  
sign(1)
```

will be translated to

RETURN

```
abs(1),  
ceil(1),  
floor(1),  
round(1),  
round(1, 1),  
sign(1)
```

Logarithmic functions

Neo4j supports a broad range of [logarithmic functions](#).

SELECT

```
exp(1),  
ln(1),  
log(2, 1),  
log10(1),  
sqrt(1)
```

will be translated to

RETURN

```
exp(1),  
log(1),  
(log(1) / log(2)),  
log10(1),  
sqrt(1)
```

Trigonometric functions

Calls to trigonometric functions

SELECT

```
acos(1),  
asin(1),  
atan(1),
```

```
atan2(1, 2),
cos(1),
cot(1),
degrees(1),
pi(),
radians(1),
sin(1),
tan(1)
```

will be translated to the corresponding [Neo4j functions](#):

```
RETURN
  acos(1),
  asin(1),
  atan(1),
  atan2(1, 2),
  cos(1),
  cot(1),
  degrees(1),
  pi(),
  radians(1),
  sin(1),
  tan(1)
```

String functions

The following string manipulations are guaranteed to work:

```
SELECT
  lower('abc'),
  cast(3 as varchar),
  trim(' abc '),
  length('abc'),
  left('abc', 2),
  ltrim(' abc '),
  replace('abc', 'b'),
  replace('abc', 'b', 'x'),
  reverse('abc'),
  right('abc', 2),
  rtrim(' abc '),
  substring('abc', 2 - 1),
  substring('abc', 2 - 1, 2),
  upper('abc')
```

and will be translated to [Neo4j's versions](#):

```
RETURN
  toLower('abc'),
```

```

toString(3),
trim(' abc '),
size('abc'),
left('abc', 2),
ltrim(' abc '),
replace('abc', 'b', NULL),
replace('abc', 'b', 'x'),
reverse('abc'),
right('abc', 2),
rtrim(' abc '),
substring('abc', (2 - 1)),
substring('abc', (2 - 1), 2),
toUpperCase('abc')

```

Scalar functions

The input

```

SELECT
  coalesce(1, 2),
  coalesce(1, 2, 3),
  nvl(1, 2),
  cast('1' as boolean),
  cast(1 as float),
  cast(1 as double precision),
  cast(1 as real),
  cast(1 as tinyint),
  cast(1 as smallint),
  cast(1 as int),
  cast(1 as bigint)

```

will be translated to (see [Scalar functions](#)):

```

RETURN
  coalesce(1, 2),
  coalesce(1, 2, 3),
  coalesce(1, 2),
  toBoolean('1'),
  toFloat(1),
  toFloat(1),
  toFloat(1),
  toInteger(1),
  toInteger(1),
  toInteger(1),
  toInteger(1)

```


Query expressions

Several advanced SQL expressions are supported as well.

CASE simple

The simple CASE expressions

```
SELECT
  CASE 1 WHEN 2 THEN 3 END,
  CASE 1 WHEN 2 THEN 3 ELSE 4 END,
  CASE 1 WHEN 2 THEN 3 WHEN 4 THEN 5 END,
  CASE 1 WHEN 2 THEN 3 WHEN 4 THEN 5 ELSE 6 END
```

```
RETURN CASE 1 WHEN 2 THEN 3 END, CASE 1 WHEN 2 THEN 3 ELSE 4 END, CASE 1 WHEN 2 THEN 3
  WHEN 4 THEN 5 END, CASE 1 WHEN 2 THEN 3 WHEN 4 THEN 5 ELSE 6 END
```

See [Cypher → Conditional expressions \(CASE\)](#) for more information.

CASE advanced

And CASE statement using a search:

```
SELECT
  CASE WHEN 1 = 2 THEN 3 END,
  CASE WHEN 1 = 2 THEN 3 ELSE 4 END,
  CASE WHEN 1 = 2 THEN 3 WHEN 4 = 5 THEN 6 END,
  CASE WHEN 1 = 2 THEN 3 WHEN 4 = 5 THEN 6 ELSE 7 END
```

will be translated to

```
RETURN
  CASE WHEN 1 = 2 THEN 3 END,
  CASE WHEN 1 = 2 THEN 3 ELSE 4 END,
  CASE WHEN 1 = 2 THEN 3 WHEN 4 = 5 THEN 6 END,
  CASE WHEN 1 = 2 THEN 3 WHEN 4 = 5 THEN 6 ELSE 7 END
```

See [Cypher → Conditional expressions \(CASE\)](#) for more information.

CASE abbreviations (which are not COALESCE or NVL)

The input

```
SELECT
  nullif(1, 2),
  nvl2(1, 2, 3)
```

will be translated to

```
RETURN
  CASE WHEN 1 = 2 THEN NULL ELSE 1 END,
  CASE WHEN 1 IS NOT NULL THEN 2 ELSE 3 END
```

6.3.6. Predicates

As with expressions a lot of logical SQL expressions and conditions used as predicates can be translated straightforward into Cypher predicates.

Conjunctions and disjunctions

Logical conjunctions and disjunctions are all supported.

```
SELECT 1 FROM p WHERE 1 = 1 AND 2 = 2 OR 3 = 3
```

```
MATCH (p:p)
WHERE ((1 = 1
      AND 2 = 2)
      OR 3 = 3)
RETURN 1
```

The input

```
SELECT 1 FROM p WHERE NOT 1 = 1 XOR 2 = 2
```

will be translated to

```
MATCH (p:p)
WHERE (NOT (1 = 1)
      XOR 2 = 2)
RETURN 1
```

Operators

Arithmetic operators

The input

```
SELECT 1 FROM p WHERE 1 = 1 AND 2 > 1 AND 1 < 2 AND 1 <= 2 AND 2 >= 1 AND 1 != 2
```

will be translated to

```
MATCH (p:p)
WHERE (1 = 1
      AND 2 > 1
      AND 1 < 2
      AND 1 <= 2
      AND 2 >= 1
      AND 1 <> 2)
RETURN 1
```

Between

Between in SQL is inclusive

```
SELECT 1 FROM p WHERE 2 BETWEEN 1 AND 3
```

and will be translated to (we can't generate the shorter form $1 \Leftarrow 2 \Leftarrow 3$) due to restrictions in the underlying generator):

```
MATCH (p:p)
WHERE (1 <= 2) AND (2 <= 3)
RETURN 1
```

SQL has a **SYMMETRIC** keyword for the **BETWEEN** clause, to indicate that you do not care which bound of the range is larger:

```
SELECT 1 FROM p WHERE 2 BETWEEN SYMMETRIC 3 AND 1
```

We translate this into a disjunction:

```
MATCH (p:p)
WHERE (3 <= 2) AND (2 <= 1) OR (1 <= 2) AND (2 <= 3)
RETURN 1
```

Logical row value expressions

The above examples are based on scalar expressions. Row value expressions will be translated as well:

```
SELECT 1
FROM p
WHERE (1, 2) = (3, 4)
```

```
OR (1, 2) < (3, 4)
OR (1, 2) <= (3, 4)
OR (1, 2, 3) <> (4, 5, 6)
OR (1, 2, 3) > (4, 5, 6)
OR (1, 2, 3) >= (4, 5, 6)
```

Leads to a semantically equivalent cypher:

```
MATCH (p:p)
WHERE 1 = 3 AND 2 = 4
OR (1 < 3 OR 1 = 3 AND 2 < 4)
OR (1 < 3 OR 1 = 3 AND 2 <= 4)
OR (1 != 4 AND 2 != 5 AND 3 != 6)
OR (1 > 4 OR 1 = 4 AND (2 > 5 OR 2 = 5 AND 3 > 6))
OR (1 > 4 OR 1 = 4 AND (2 > 5 OR 2 = 5 AND 3 >= 6))
RETURN 1
```

Null Handling

For scalar expressions

The input

```
SELECT 1 FROM p WHERE 1 IS NULL AND 2 IS NOT NULL
```

will be translated to

```
MATCH (p:p)
WHERE (1 IS NULL
      AND 2 IS NOT NULL)
RETURN 1
```

For row value expressions

The input

```
SELECT 1 FROM p WHERE (1, 2) IS NULL OR (3, 4) IS NOT NULL
```

will be translated to

```
MATCH (p:p)
WHERE
  (1 IS NULL AND 2 IS NULL)
  OR (3 IS NOT NULL AND 4 IS NOT NULL)
```

```
RETURN 1
```

LIKE operator

The **LIKE** operator

```
SELECT * FROM movies m WHERE m.title LIKE '%Matrix%' OR m.title LIKE 'M_trix'
```

will be translated into a regular expressions, replacing the % with .*:

```
MATCH (m:`movies`) WHERE m.title CONTAINS 'Matrix' OR m.title =~ 'M.trix'
RETURN *
```

6.3.7. Using joins to map relationships

On the surface, joins are relationships materialized in SQL (foreign keys are not). Sadly, it's not as straightforward to map to Cypher. There are several implementation options:

- When joining two tables on a column, take the left hand table column, use its name as relationship type, and treat it as outgoing from left to right.
- When joining two tables with an intersection table (which it's usually modeled in SQL for **m:n** relationships with attributes), use the name of that intersection table as relationship type.

We implemented some variants thereof, however we don't claim their absolute usefulness in all cases.



Outer joins are not supported.

1:n joins

Natural joins

SQL **NATURAL** joins are the easiest way to denote relationship names without having to do any mapping. A one-hope **NATURAL JOIN** will translate to an anonymous, wildcard relationship.

```
SELECT p, m FROM Person p
NATURAL JOIN Movie m
```

```
MATCH (p:Person)-->(m:Movie) RETURN p, m
```

NATURAL joins can be chained, and the connecting join table does not need to exist. This will be turned into a Neo4j relationship:

```
SELECT p.name, r.roles, m.* FROM Person p
NATURAL JOIN ACTED_IN r
NATURAL JOIN Movie m
```

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
RETURN p.name, r.roles,
       elementId(m) AS `v$id`, m.title AS title, m.released AS released
```

Simple join

Assume we configured the translator to use the following table mapping:

- **people** mapped to label **People**
- **movies** mapped to label **Movie**

With that in place, we translate

```
SELECT p.name, m.title
FROM people p
JOIN movies m ON m.id = p.directed
```

to

```
MATCH (p:Person)-[directed:DIRECTED]->(m:Movie)
RETURN p.name, m.title
```

DIRECTED is the uppercase version of the join column in the left table (**p.directed**).

We can add a join column mapping in the form of **people.movie_id:DIRECTED** if we have different column names:

```
SELECT p.name, m.title
FROM people p
JOIN movies m ON m.id = p.movie_id
```

to

```
MATCH (p:Person)-[directed:DIRECTED]->(m:Movie)
RETURN p.name, m.title
```

Using the **ON** clause

We used backticks here for the table and column names and no mapping.

```
SELECT p.name, m.title
FROM `Person` as p
JOIN `Movie` as m ON (m.id = p.`DIRECTED`)
```

The translation is same as before:

```
MATCH (p:Person)-[directed:DIRECTED]->(m:Movie)
RETURN p.name, m.title
```

m:n joins

An intersection table is a table that contains references to two other tables in the form of at least two columns. This construct is usually required in the relational model to create an **m:n** relationship. Such an auxiliary construct is not necessary in Neo4j. We can model as many outgoing and incoming relationships from one label to another, and they can also have properties. We can thus use that construct for our translator.

The following example uses a configured mapping as follows:

- **people** mapped to label **People**
- **movies** mapped to label **Movie**
- **movie_actors** mapped to **ACTED_IN**

```
SELECT p.name, m.title
FROM people p ①
JOIN movie_actors r ON r.person_id = p.id ②
JOIN movies m ON m.id = r.person_id ③
```

- ① The table from which to map outgoing relationships
- ② An intersection table, that is used again in the next **JOIN** clause
- ③ The final **JOIN** clause

We do no semantic analysis: the order of the joins matter, and will lead to the following query:

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
RETURN p.name, m.title
```

Multiple joins will result in a chain of relationships:

```
SELECT p.name AS actor, d.name AS director, m.title
FROM people p
JOIN movie_actors r ON r.person_id = p.id
JOIN movies m ON m.id = r.person_id
JOIN movie_directors r2 ON r2.movie_id = m.id
```

```
JOIN people d ON r2.person_id = d.id
```

```
MATCH (p:`Person`)-[r:`ACTED_IN`]->(m:`Movie`)<-[r2:`DIRECTED`]->(d:`Person`)
RETURN p.name AS actor, d.name AS director, m.title
```

Notice how the direction of the **DIRECTED** relationship is defined by the order of the join columns.

6.3.8. Using aggregates and **GROUP BY**

Cypher uses implicit grouping by default. Cypher 5 does not have explicit grouping and will not gain it; as of March 2026, explicit grouping is being developed for Cypher 25.

For the common case (**GROUP BY** columns matching the projected columns), the translator continues to emit simple **MATCH ... RETURN** statements because Cypher's implicit grouping produces the correct result. When the **GROUP BY** contains columns absent from the projection, or when a **HAVING** clause is present, the translator introduces a Cypher **WITH** clause to make grouping explicit and to support post-aggregation filtering.

The following functionality is supported:

- **GROUP BY**: implicit and explicit grouping
- **HAVING**: simple conditions and compound conditions
- **ORDER BY** on aggregate aliases
- **ORDER BY** after **WITH** clause
- **DISTINCT** with **GROUP BY** / **HAVING**
- **LIMIT** and **OFFSET** with **WITH** clauses
- **WHERE** and **GROUP BY** combinations
- **JOIN** and **GROUP BY**: aggregation across relationships, with and without **WITH** clauses

WITH clause generation

SQL	Cypher equivalent
SELECT name, sum(age) FROM People p GROUP BY name, department	MATCH (p:People) WITH p.name AS name, sum(p.age) AS `sum(age)`, p.department AS __group_col_0 RETURN name, `sum(age)`
SELECT name AS x, sum(age) AS y FROM People p GROUP BY x, department	MATCH (p:People) WITH p.name AS x, sum(p.age) AS y, p.department AS __group_col_0 RETURN x, y
SELECT count(*) FROM Customers c JOIN Orders o ON c.id = o.customer_id GROUP BY c.name	MATCH (c:Customers)<-[customer_id:CUSTOMER_ID]-(o:Orders) WITH count() AS `count()`, c.name AS __group_col_0 RETURN `count(*)`
SELECT c.name, count(*) FROM Customers c JOIN Orders o ON c.id = o.customer_id GROUP BY c.name	MATCH (c:Customers)<-[customer_id:CUSTOMER_ID]-(o:Orders) RETURN c.name, count(*)

SQL	Cypher equivalent
SELECT name, count(*), sum(age), avg(age) FROM People p GROUP BY name	MATCH (p:People) RETURN p.name AS name, count(*), sum(p.age), avg(p.age)
SELECT count(*), sum(age) FROM People p GROUP BY name	MATCH (p:People) WITH count() AS `count()`, sum(p.age) AS `sum(age)`, p.name AS __group_col_0 RETURN `count(*)`, `sum(age)`
SELECT sum(age) FROM People p GROUP BY name HAVING sum(age) > 100	MATCH (p:People) WITH sum(p.age) AS `sum(age)`, p.name AS __group_col_0 WHERE `sum(age)` > 100 RETURN `sum(age)`
SELECT sum(age) FROM People p GROUP BY name ORDER BY sum(age)	MATCH (p:People) WITH sum(p.age) AS `sum(age)`, p.name AS __group_col_0 RETURN `sum(age)` ORDER BY `sum(age)`

Filtering aggregates with the **HAVING** clause

SQL	Cypher equivalent
SELECT name FROM People p GROUP BY name HAVING count(*) > 5	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0 WHERE having_col_0 > 5 RETURN name
SELECT count() FROM People p HAVING count() > 5	MATCH (p:People) WITH count() AS `count()` WHERE `count()` > 5 RETURN `count()`
SELECT count(*) FROM People p GROUP BY name HAVING name = 'Alice'	MATCH (p:People) WITH count() AS `count()`, p.name AS group_col_0 WHERE group_col_0 = 'Alice' RETURN `count(*)`
SELECT name, count(*) AS cnt FROM People p GROUP BY name HAVING cnt > 5	MATCH (p:People) WITH p.name AS name, count(*) AS cnt WHERE cnt > 5 RETURN name, cnt
SELECT name FROM People p GROUP BY name HAVING count(*) > 5 AND max(age) > 50	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0, max(p.age) AS having_col_1 WHERE (having_col_0 > 5 AND having_col_1 > 50) RETURN name
SELECT name, sum(age) FROM People p GROUP BY name HAVING sum(age) > 100 AND count(*) > 2	MATCH (p:People) WITH p.name AS name, sum(p.age) AS `sum(age)`, count(*) AS having_col_0 WHERE (`sum(age)` > 100 AND having_col_0 > 2) RETURN name, `sum(age)`
SELECT name FROM People p GROUP BY name HAVING count(*) > 5 ORDER BY name	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0 WHERE having_col_0 > 5 RETURN name ORDER BY name
SELECT name FROM People p GROUP BY name HAVING count(DISTINCT age) > 3	MATCH (p:People) WITH p.name AS name, count(DISTINCT p.age) AS having_col_0 WHERE having_col_0 > 3 RETURN name

Ordering the result set by an aggregate

SQL	Cypher equivalent
SELECT name, count(*) AS cnt FROM People p GROUP BY name ORDER BY cnt	MATCH (p:People) RETURN p.name AS name, count(*) AS cnt ORDER BY cnt
SELECT name, count(*) AS cnt FROM People p GROUP BY name ORDER BY cnt DESC	MATCH (p:People) RETURN p.name AS name, count(*) AS cnt ORDER BY cnt DESC
SELECT department, sum(age) AS total FROM People p GROUP BY department ORDER BY total	MATCH (p:People) RETURN p.department AS department, sum(p.age) AS total ORDER BY total
SELECT department, count(*) AS cnt, avg(age) AS average FROM People p GROUP BY department ORDER BY cnt DESC, average	MATCH (p:People) RETURN p.department AS department, count(*) AS cnt, avg(p.age) AS average ORDER BY cnt DESC, average

Supported aggregate functions

SQL	Cypher equivalent
SELECT name, count(*) FROM People p GROUP BY name	MATCH (p:People) RETURN p.name AS name, count(*)
SELECT name, max(age) FROM People p GROUP BY name	MATCH (p:People) RETURN p.name AS name, max(p.age)
SELECT name, min(age) FROM People p GROUP BY name	MATCH (p:People) RETURN p.name AS name, min(p.age)
SELECT sum(age) FROM People p GROUP BY name	MATCH (p:People) WITH sum(p.age) AS `sum(age)\`, p.name AS __group_col_0 RETURN `sum(age)\` ``
SELECT avg(age) FROM People p GROUP BY name	MATCH (p:People) WITH avg(p.age) AS `avg(age)\`, p.name AS __group_col_0 RETURN `avg(age)\` ``
SELECT percentileCont(age) FROM People p GROUP BY name	MATCH (p:People) WITH percentileCont(p.age) AS `percentileCont(age)\`, p.name AS __group_col_0 RETURN `percentileCont(age)\` ``
SELECT percentileDisc(age) FROM People p GROUP BY name	MATCH (p:People) WITH percentileDisc(p.age) AS `percentileDisc(age)\`, p.name AS __group_col_0 RETURN `percentileDisc(age)\` ``
SELECT stDev(age) FROM People p GROUP BY name	MATCH (p:People) WITH stDev(p.age) AS `stddev_samp(age)\`, p.name AS __group_col_0 RETURN `stddev_samp(age)\` ``
SELECT stDevP(age) FROM People p GROUP BY name	MATCH (p:People) WITH stDevP(p.age) AS `stddev_pop(age)\`, p.name AS __group_col_0 RETURN `stddev_pop(age)\` ``
SELECT stDev(age) AS x FROM People p GROUP BY name	MATCH (p:People) WITH stDev(p.age) AS x, p.name AS __group_col_0 RETURN x
SELECT stDevP(age) AS x FROM People p GROUP BY name	MATCH (p:People) WITH stDevP(p.age) AS x, p.name AS __group_col_0 RETURN x

Limit and offset are supported with aggregations

SQL	Cypher equivalent
SELECT sum(age) FROM People p GROUP BY name LIMIT 5	MATCH (p:People) WITH sum(p.age) AS `sum(age)\`, p.name AS __group_col_0 RETURN `sum(age)\` LIMIT 5`
SELECT name FROM People p GROUP BY name HAVING count(*) > 5 LIMIT 10	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0 WHERE having_col_0 > 5 RETURN name LIMIT 10
SELECT name FROM People p GROUP BY name HAVING count(*) > 5 ORDER BY name LIMIT 10 OFFSET 2	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0 WHERE having_col_0 > 5 RETURN name ORDER BY name SKIP 2 LIMIT 10

DISTINCT in the context of GROUP BY and HAVING

DISTINCT applies to the final RETURN, not the WITH, across all GROUP BY / HAVING paths.

SQL	Cypher equivalent
SELECT DISTINCT name, count(*) FROM People p GROUP BY name	MATCH (p:People) RETURN DISTINCT p.name AS name, count(*)
SELECT DISTINCT count(*) FROM People p GROUP BY name	MATCH (p:People) WITH count() AS `count()`, p.name AS __group_col_0 RETURN DISTINCT `count()``
SELECT DISTINCT name FROM People p GROUP BY name HAVING count(*) > 5	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0 WHERE having_col_0 > 5 RETURN DISTINCT name

All supported clauses combined

SQL	Cypher equivalent
SELECT DISTINCT name, sum(age) FROM People p GROUP BY name, department HAVING sum(age) > 100 ORDER BY name LIMIT 5	MATCH (p:People) WITH p.name AS name, sum(p.age) AS `sum(age)\`, p.department AS __group_col_0 WHERE `sum(age)\` > 100 RETURN DISTINCT name, `sum(age)\` ORDER BY name LIMIT 5`
SELECT DISTINCT name FROM People p GROUP BY name HAVING count(*) > 5 ORDER BY name LIMIT 10 OFFSET 5	MATCH (p:People) WITH p.name AS name, count(*) AS having_col_0 WHERE having_col_0 > 5 RETURN DISTINCT name ORDER BY name SKIP 5 LIMIT 10
SELECT DISTINCT department, count() AS cnt, max(age) AS max_age FROM People p WHERE age > 18 GROUP BY department HAVING count() > 1 AND max(age) > 25 ORDER BY cnt DESC LIMIT 10 OFFSET 2	MATCH (p:People) WHERE p.age > 18 WITH p.department AS department, count(*) AS cnt, max(p.age) AS max_age WHERE (cnt > 1 AND max_age > 25) RETURN DISTINCT department, cnt, max_age ORDER BY cnt DESC SKIP 2 LIMIT 10
SELECT department, count(*) AS cnt FROM People p WHERE age > 18 GROUP BY department	MATCH (p:People) WHERE p.age > 18 RETURN p.department AS department, count(*) AS cnt
SELECT count(*) AS cnt FROM People p WHERE age > 18 GROUP BY department	MATCH (p:People) WHERE p.age > 18 WITH count(*) AS cnt, p.department AS __group_col_0 RETURN cnt
SELECT count(*) AS cnt FROM People p ORDER BY cnt	MATCH (p:People) RETURN count(*) AS cnt ORDER BY cnt

6.3.9. DML statements

This section lists the supported Data-Manipulation-Language (DML) statements. Although a **SELECT** statement is technically DML as well, it is covered in [\[^top\]](#).

Deleting nodes

Nodes can be deleted via the SQL **DELETE** statement.

For example, to unconditionally delete all **person** nodes:

```
DELETE FROM person
```

```
MATCH (person:person)
DELETE person
```

A **WHERE** clause can be added to prevent this:

```
DELETE FROM person
WHERE person.id = 1
```

```
MATCH (person:person)
WHERE person.id = 1
DELETE person
```

If you want to delete everything, but your tooling complains, just add a conditions that is always **true**:

```
DELETE FROM person
WHERE true
```

```
MATCH (person:person)
WHERE true
DELETE person
```

Of, the condition can also be that always evaluates to **false**, never deleting anything:

```
DELETE FROM person
WHERE false
```

```
MATCH (person:person)
WHERE false
DELETE person
```

Tables can be aliased, and the alias will be used in Cypher, too:

```
DELETE FROM person p
```

```
MATCH (p:person)
DELETE p
```

Aliasing tables is also supported in combination with specifying the label to which the table name is mapped. Using the same query with `table_mappings=person:Person` configured,

```
DELETE FROM person p
```

will be translated to

```
MATCH (p:Person)
DELETE p
```

Deleting nodes and their related nodes

You can use SQL `TRUNCATE` to `DETACH DELETE` nodes.

```
TRUNCATE TABLE people
```

will be translated to

```
MATCH (people:Person)
DETACH DELETE people
```

Inserting data

A single list of values with explicit columns and constant values can be inserted with a simple `INSERT` statement:

```
INSERT INTO People (first_name, last_name, born) VALUES ('Helge', 'Schneider', 1955)
```

```
CREATE (people:`Person` {first_name: 'Helge', last_name: 'Schneider', born: 1955})
```

All expressions, including parameters, are supported. Parameters will be named from 1 on upwards in Cypher.

```
INSERT INTO People (first_name, last_name, born) VALUES (?, ?, ?)
```

```
CREATE (people:`Person` {first_name: $1, last_name: $2, born: $3})
```

If you omit the column names on the insertion target, we generate names:

```
INSERT INTO People VALUES ('Helge', 'Schneider', 1955)
```

Note the **unknown field xxx** property names:

```
CREATE (people:`Person` {'unknown field 0': 'Helge', 'unknown field 1': 'Schneider',  
  'unknown field 2': 1955})
```

The SQL **VALUES** clause actually supports lists of values:

```
INSERT INTO People (first_name, last_name, born) VALUES  
  ('Helge', 'Schneider', 1955),  
  ('Bela', 'B', 1962)
```

Those values will be translated into a Cypher array to be unwound in the Cypher statement. This is a great solution for batching inserts:

```
UNWIND [  
  {first_name: 'Helge', last_name: 'Schneider', born: 1955},  
  {first_name: 'Bela', last_name: 'B', born: 1962}]  
AS properties  
CREATE (people:`Person`)  
SET people = properties
```

A returning clause is supported as well:

```
INSERT INTO People p (name) VALUES (?) RETURNING elementId(p)
```

```
CREATE (p:Person {name: $1}) RETURN elementId(p)
```

Upserts

We support a restricted range of "upserts" via the non-standard but pretty common **ON DUPLICATE** and **ON CONFLICT** SQL clauses. Upserts are translated into **MERGE** statements. While they work without constraints, you should really have uniqueness-constraints on the node properties you merge on, or Neo4j may create duplicates (see [Understanding how merge works](#)).

Upserts on all columns can happen via either **ON DUPLICATE KEY IGNORE** or **ON CONFLICT IGNORE**. While **ON DUPLICATE KEY** does offer upgrade options, it assumes the primary (or unique) key being violated to be known. Although this is most certainly the case in a relational system, this translation layer that runs without a database connection doesn't know.

*Listing 39. Upsert with **ON DUPLICATE KEY IGNORE***

```
INSERT INTO Movie(title, released) VALUES(?, ?) ON DUPLICATE KEY IGNORE
```

```
MERGE (movie:`Movie` {title: $1, released: $2})
```

*Listing 40. Upsert with **ON CONFLICT IGNORE***

```
INSERT INTO actors(name, firstname) VALUES(?, ?) ON CONFLICT DO NOTHING
```

```
MERGE (actors:`Actor` {name: $1, firstname: $2})
```

If you want to define an action, you must use **ON CONFLICT** and specify the key you want to merge on.

```
INSERT INTO tbl(i, j, k) VALUES (1, 40, 700)  
ON CONFLICT (i) DO UPDATE SET j = 0, k = 2 * EXCLUDED.k
```

Note how the special reference **EXCLUDED** can be used to refer to the values of columns that have not been part of the key. They will be reused with their values in the **ON MATCH SET** clause.

```
MERGE (tbl:`tbl` {i: 1})  
ON CREATE SET tbl.j = 40, tbl.k = 700  
ON MATCH SET tbl.j = 0, tbl.k = (2 * 700)
```

This works with parameters, too:

```
INSERT INTO tbl(i, j, k) VALUES (1, 2, ?)
ON CONFLICT (i) DO UPDATE SET j = EXCLUDED.k
```

```
MERGE (tbl:`tbl` {i: 1})
ON CREATE SET tbl.j = 2, tbl.k = $1
ON MATCH SET tbl.j = $1
```

It's possible to just specify a concrete merge column instead of merging on all columns as well. It will be translated with **ON CREATE**:

```
INSERT INTO tbl(i, j, k) VALUES (1, 40, 700)
ON CONFLICT (i) DO NOTHING
```

```
MERGE (tbl:`tbl` {i: 1})
ON CREATE SET tbl.j = 40, tbl.k = 700
```

Using **ON CONFLICT** and specifying a key is the only way to insert multiple rows with a **MERGE** statement:

```
INSERT INTO People (first_name, last_name, born) VALUES
  ('Helge', 'Schneider', 1955),
  ('Bela', 'B', 1962)
ON CONFLICT(last_name) DO UPDATE SET born = EXCLUDED.born
```

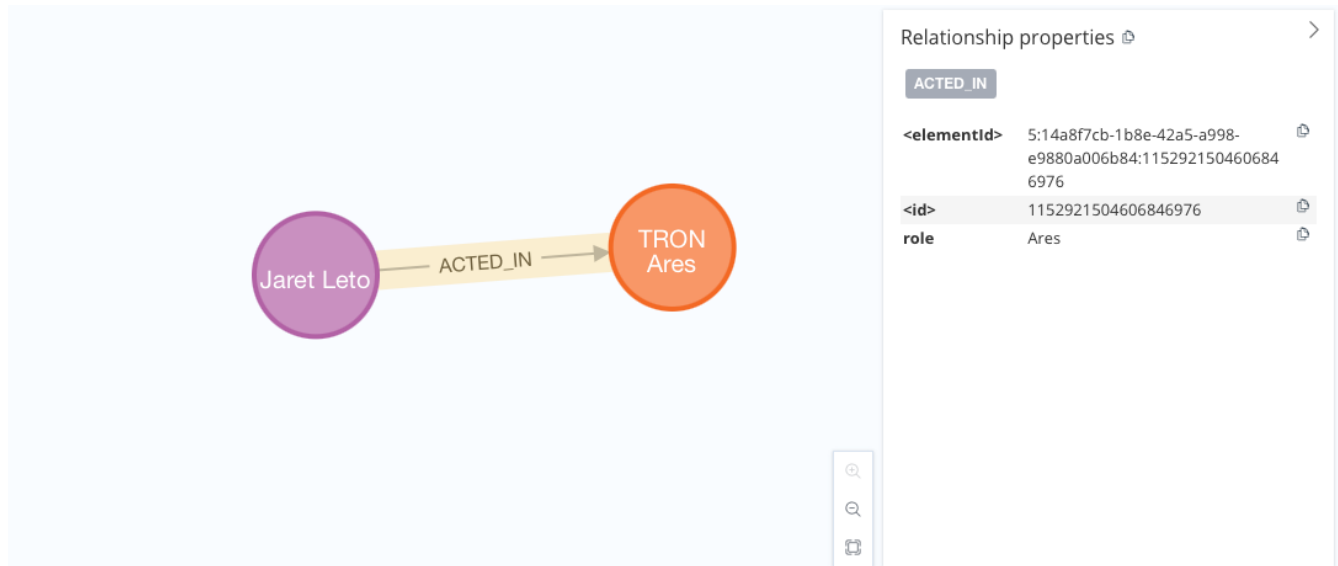
```
UNWIND [{first_name: 'Helge', last_name: 'Schneider', born: 1955}, {first_name:
'Bela', last_name: 'B', born: 1962}] AS properties
MERGE (people:`People` {last_name: properties['last_name']})
ON CREATE SET
  people.first_name = properties.first_name,
  people.born = properties.born
ON MATCH SET people.born = properties['born']
```

6.3.10. Manipulating relationships

Relationships in Neo4j involve three entities: A start node, the relationship itself and an end node. Using Cypher, this can be expressed in one statement:


```
CREATE (a:Person {name: 'Jaret Leto'})
-[:ACTED_IN {role: 'Ares'}]->
(m:Movie {title: 'TRON Ares'})
RETURN *;
```

Which gives you this graph:



In SQL on the other hand, we would normally use at least two or depending on the level of normalisation, three tables to store this (Think **people**, **movies** and **acted_in** as intersection table, referencing both **people** and **movies** and storing any properties).

While you can obviously **SELECT** from as many relations as you can think of, the **INSERT** clause in SQL does not support multiple targets of insertion, only a single table. Without such a construct, the SQL to Cypher translator cannot directly translate a SQL statement into a pattern like the one above. The translator can only try to infer an intention here on best effort base.

Inferring relationships from tables based on existing data

Assuming the above statement was executed, the JDBC driver will actually show that relationship as virtual table in the database metadata:

The screenshot shows the DBeaver 25.2.2 interface. The 'Database Navigator' on the left shows a tree view with 'Person_ACTED_IN_Movie' selected. The 'Properties' tab on the right shows the table's metadata:

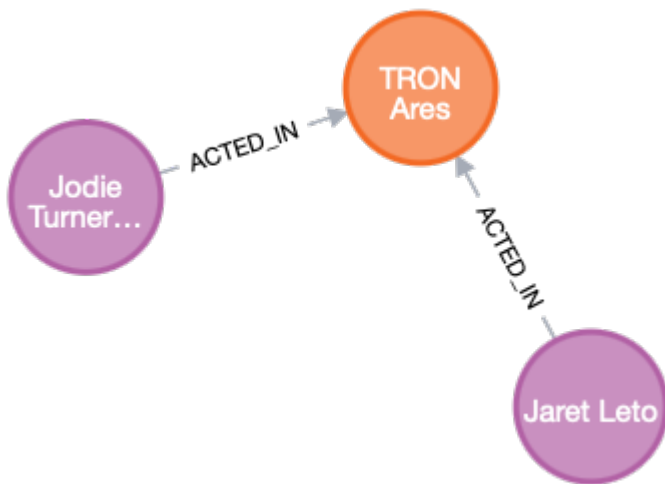
Table Name	Table Description	Table Type	Database Name
Person_ACTED_IN_Movie	Person ACTED_IN	RELATIONSHIP	neo4j

Column Name	#	Data Type	Length	Scale	Not Null	Auto Generated	Auto Increment	Default	Description
A: v\$movie_		STRING			[v]	[v]	[]		
A: v\$person_		STRING			[v]	[v]	[]		
A: v\$id		STRING			[v]	[v]	[]		
A: role	1	STRING			[]	[]	[]		

Having one relationship as this is enough for the JDBC driver to use it as a kind of template. The JDBC driver will query for a relationship-virtual-table based on the given table name and if that matches, automatically infer that the following SQL statement does target a relationship:

```
INSERT INTO Person_ACTED_IN_Movie(name, role, title)
VALUES('Jodie Turner-Smith', 'Athena', 'TRON Ares');
```

The updated graph looks like this:



Noteable things are

- No additional **Movie** node has been created
- The attributes **name** has been used with the **Person** label and **title** with the **Movie**
- **role** has been used on the relationship

The matching has been based on the virtual table. If both nodes had had a **name** attribute, the property would have been used for the start node only. Columns that match neither properties of the nodes nor the relationship, will be added to the relationship.



If the schema spots the same property name on nodes and relationships, the nodes are prioritized. If the same property appears on both sides of the relationship, it will be only used on the first appearance (the start node). If this does not work for you, you must qualify the target properties.

The **INSERT** statement also was not translated into a plain **CREATE ...** statement as shown above. Inserting into a relationship expresses a pretty clear intention to create the relationship alone, hence the JDBC driver will always create a statement such as this, if possible:

```
MERGE (lhs:LabelLeft {propN: 'Value'})
MERGE (rhs:LabelRight {propM: 'Value'})
CREATE (lhs)-[:TYPE]->(rhs);
```

All properties for the nodes will be used for merging. If the JDBC driver cannot match any property to a label, it will issue a **CREATE** statement for that label.



If you can, make sure you define a unique constraint on the properties of the nodes on which you are going to insert relationships, for performance reasons, but

also to account for Neo4js **MERGE** behaviour. Under concurrent updates, **MERGE** only guarantees the existence of the **MERGE** pattern, but not uniqueness, see [MERGE](#).

This works the same way with a multiple values. A statement like the following:

```
INSERT INTO Person_ACTED_IN_Movie(name, role, title) VALUES
('Gillian Anderson', 'Elisabeth Dillinger', 'TRON Ares'),
('Arturo Castro', 'Seth Flores', 'TRON Ares');
```

or a batched JDBC variant such as follows:

```
try (var statement = connection.prepareStatement("""
    INSERT INTO Person_ACTED_IN_Movie(name, role, title)
    VALUES (?, ?, ?)"""))
{
    statement.setString(1, "Jeff Bridges");
    statement.setString(2, "Kevin Flynn");
    statement.setString(3, "TRON Ares");
    statement.addBatch();
    statement.setString(1, "Greta Lee");
    statement.setString(2, "Eve Kim");
    statement.setString(3, "TRON Ares");
    statement.executeBatch();
}
```

Will be translated into a Cypher query utilizing **UNWIND** and thus being great for batch loading relationships.

Inferring relationships based on the table name

The translator is by default configured in such a way that it tries to infer a relationship from the name of the table that should be inserted into if there's neither a matching existing relationship nor a node with the same label:

Target table of the **INSERT** clause is: **Person_ACTED_IN_Movie**:

- A relationship **(:Person)-[:ACTED_IN]→(:Movie)** exists: This relationship and the properties of all its constituents will be used as template
- A node with the label **Person_ACTED_IN_Movie** exists: **INSERT** clause will be translated into **CREATE (n:Person_ACTED_IN_Movie)**, no attempt of creating relationships will be made
- Otherwise, as **Person_ACTED_IN_Movie** matches the default pattern nodes labeled **Person** and **Movie** will either be merged or created, and a new relationship **ACTED_IN** will be created

The default pattern to infer a relationship is designed in such a way that it detects labels in mixed-case letters that might contain underscores as well separated by a single underscore on each site from a relationship type in upper case, i.e. **Person_ACTED_IN_Movie** will be detected as well as

A_RELATES_TO_B. The latter will use labels **A** and **B** for nodes, and **RELATES_TO** for the relationship type.

Without an existing template, we need a way to map properties to the source or the target node or the relationship. By default, all properties will be put onto the relationship when the relationship has been inferred by just the table name.

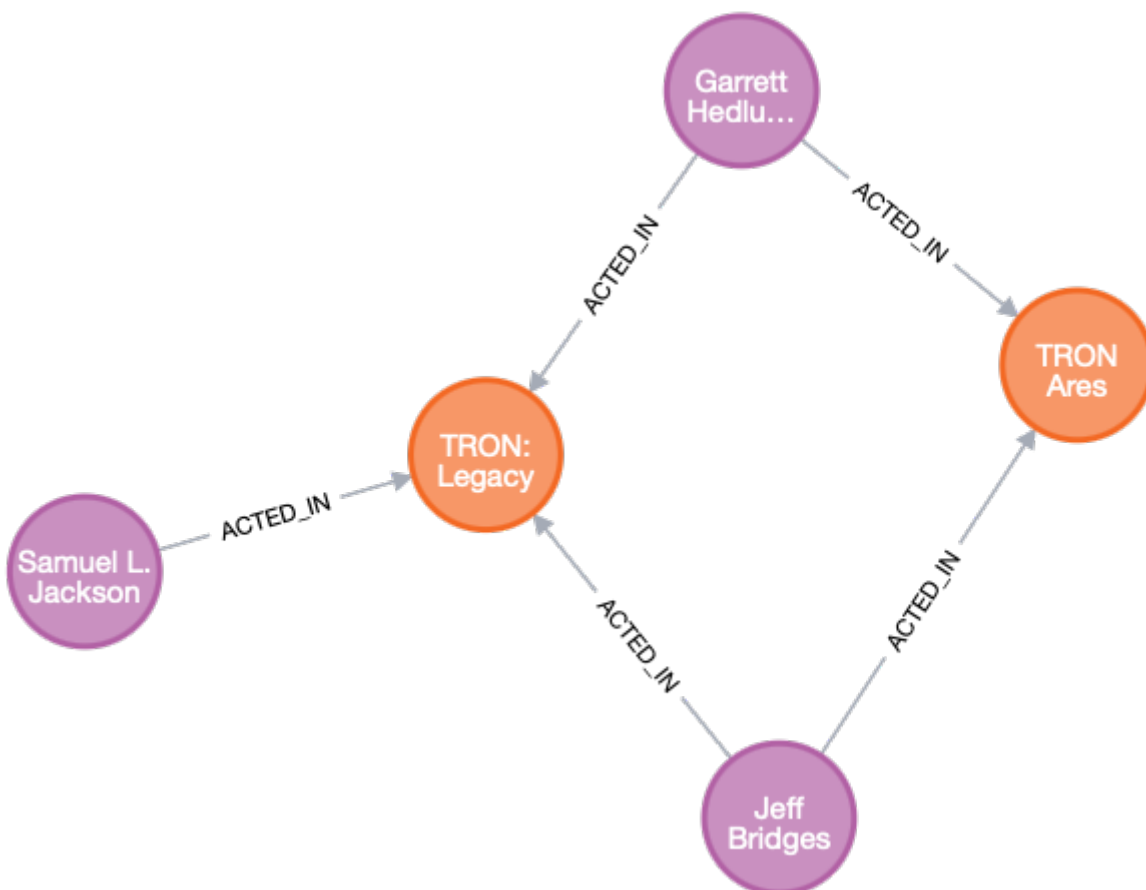
The JDBC driver however can utilize a SQL feature that lets you qualify columns like this:

```
INSERT INTO Person_ACTED_IN_Movie(Person.name, ACTED_IN.role, Movie.title)
VALUES ('Evan Peters', 'Julian Dillinger', 'TRON Ares');
```

Now, most SQL databases won't support qualified columns in an **INSERT** statement, but it's valid SQL, and we can parse it. The above qualified columns will be appropriately distributed onto **Person**, **Movie** and **ACTED_IN**.

Deleting relationships

While Samuel L. Jackson did play in a lot of movies, he didn't play himself in **TRON: Legacy** and you might want to delete that relationship from the Graph:



This is possible with an ordinary **DELETE FROM** statement on which the inference of relationships works the same way as described above. Aligned with the start and end nodes being merged if possible during create and not modified otherwise, the deletion will only affect the relationship itself:

```
DELETE FROM Person_ACTED_IN_Movie
WHERE title = 'TRON: Legacy'
  AND name = 'Samuel L. Jackson'
  AND role = 'himself'
```

Predicates will be pushed down in such a way that they filter on the correct target (here: The **name** property of the **:Actor** node, **title** on **:Movie** and **role** on the relationship **:ACTED_IN**). The result can be confirmed with Cypher (the JDBC driver does not support outer joins):

```
/*+ NEO4J FORCE_CYPHER */
MATCH (p:Person {name: 'Samuel L. Jackson'})
RETURN p.name, COUNT {(p)-[:ACTED_IN]->(:Movie)} AS cnt
```

The result will be one row with stating that Samuel L. Jackson played in zero movies, which is obviously a true lie.

Truncating relationships

The **TRUNCATE** keyword works the same way, except that as per SQL standard, it will unconditionally delete all relationship instances:

```
TRUNCATE Person_ACTED_IN_Movie
```

will delete all **ACTED_IN** relationships, but none of the attached nodes.

Updating relationships

When you create a graph like that

```
INSERT INTO Person_ACTED_IN_Movie(Person.name, ACTED_IN.role, Movie.title)
VALUES
  ('Jaret Leto', 'Ares', 'Morbius'),
  ('Greta Lee', 'Eve Kim', 'TRON Ares'),
  ('Jodie Turner-Smith', 'Elisha James', 'TRON Ares')
```

realising later that Jodie Turner-Smith played Athena, you can fix that relationship via an **UPDATE** statement:

```
try (var stmt = con.prepareStatement(
  "UPDATE Person_ACTED_IN_Movie SET role = ? WHERE name = ?")
) {
  stmt.setString(1, "Athena");
  stmt.setString(2, "Jodie Turner-Smith");
  stmt.executeUpdate();
}
```

The same algorithm that is used to determine which column is applied to start-node, relationship or end-node properties during insert is applied for the update. That means you can also update properties of nodes. But be aware that those updates will only be applied via **SET** and no other **MERGE** updates are made. That means fixing the second bug in the above graph—making sure Leto played Ares in TRON Ares, not in Morbius—creates a **second** **:Movie** node with the name **TRON Ares**:

```
UPDATE Person_ACTED_IN_Movie
SET title = 'TRON Ares'
WHERE name = 'Jared Leto'
```

will result with two **:Movie**-Nodes labeled **TRON Ares** (**SELECT count(*) FROM Movie WHERE title = 'TRON Ares'** will return 2).

7. Cypher-backed views

7.1. Introduction

Cypher-backed views are a way to create virtual views inside the JDBC client application that are backed by an arbitrary Cypher query. They allow your Neo4j database to be utilized the best way possible and make use of all Cypher constructs, including everyone for which the JDBC driver can offer a direct SQL to Cypher translation.

Cypher-backed views will appear in the database metadata of the JDBC connection and are defined on the client-side. They consist of

- a name
- a Cypher query
- the list of columns

The name must be unique and will be the name under which the view is queryable and appear in the database metadata. Only the enumerated columns will appear in the database metadata.

7.2. Requirements

Cypher-backed views requires [SQL translations](#) to be enabled and [the default translator](#) on the classpath, hence you must add the corresponding dependency in your application or use [the full bundle](#). If you fail to enable SQL translation or are missing the standard translator, your Cypher-backed views will still be loaded, contribute to the database metadata, but you cannot query them.



While you can use any valid Cypher query in a view definition, we highly recommend not using Cypher statements that do updates to the graph. We don't parse the Cypher, and we won't prevent a **SELECT *** statement that casually deletes your graph with a view definition such as **MATCH (n) DETACH DELETE**. We do prevent Cypher-backed views from being used in a SQL update statements, such as **INSERT**, **DELETE** or **UPDATE**, though.

The Cypher query can basically have any shape, but it must be a query that is valid inside a `CALL {}` subquery. While we could just execute the query as-is, you wouldn't be able to query more than one Cypher-backed view at a time otherwise. Check ["Restrictions"](#) below for more information.

7.3. Configuration and view definition format

Cypher-backed views are represented as JSON. The native format is defined by the following [schema](#):

Listing 41. JSON Schema for Cypher-backed views

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "JSON Schema for Cypher-backed views",
  "type": "array",
  "items": {
    "$ref": "#/$defs/viewType"
  },
  "$defs": {
    "viewType": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "The name of this view."
        },
        "query": {
          "type": "string",
          "description": "Any valid Cypher query."
        },
        "columns": {
          "type": "array",
          "items": {
            "$ref": "#/$defs/columnType"
          }
        }
      },
      "required": [
        "name",
        "query",
        "columns"
      ],
      "title": "A view that will be queryable."
    },
    "columnType": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "The name as it will appear in the column list of the view."
        }
      }
    }
  }
}
```

```

    },
    "propertyName": {
      "type": "string",
      "description": "The property to pick from the graph, can be omitted when
equal to the column name"
    },
    "type": {
      "type": "string",
      "description": "The Neo4j type of this column"
    }
  },
  "required": [
    "name",
    "type"
  ],
  "description": "A type representing a column in a relational view"
}
}
}

```

The JDBC driver is able to load resources from

- the file system
- or via http(s)

and the resources must be configured through a connection property named `viewDefinitions`. That property goes either as a query parameter into the JDBC URL or as a JDBC property. Here are a bunch of examples that uses a query parameter as part of a JDBC URL:



Please be aware that we are strict with [File URI scheme](#) with regard to the number of slashes: `file:` is the protocol, the slashes are not part of it. Either use exactly one slash (/) meaning no hostname, or three slashes (///) meaning an empty hostname (effectively the same) after it. We don't guess about the meaning of two slashes and don't support hostnames for reading files.

`jdbc:neo4j://localhost:7687?viewDefinitions=file:/my/views/foobar.json`

Here the view definition resides in a folder `/my/views/` on the same machine named `foobar.json`

`jdbc:neo4j://localhost:7687?viewDefinitions=file:///my/views/foobar.json`

This is equivalent to the URL above

`jdbc:neo4j://localhost:7687?viewDefinitions=https://myorg.com/movies.json`

View definitions hosted on a secure webserver



Make sure you only load view definitions from trusted sources. As stated above, there's nothing that prevents a writing Cypher query being used as a view definition and you don't want to have them injected into your application from an unknown or untrusted source.

A simple example looks like this:

Listing 42. A Cypher-backed view that returns all movies and their actors.

```
[
  {
    "name": "v_movie_actors",
    "query": "MATCH (n:Movie)<-[:ACTED_IN]-(p:Person) RETURN elementId(n) AS id,
n.title AS title, n.released AS released, collect(p.name) AS actors",
    "columns": [
      {
        "name": "id",
        "type": "STRING"
      },
      {
        "name": "title",
        "type": "STRING"
      },
      {
        "name": "released",
        "type": "INTEGER"
      },
      {
        "name": "actors",
        "type": "LIST"
      }
    ]
  }
]
```

In the above example, all columns are projected in the Cypher **RETURN** clause and named to match the column-names in the list of columns to follow via **AS**. The view can be queried as follows, with ordering and predicates being pushed down into the actual query:

Listing 43. Querying v_movie_actors

```
SELECT * FROM v_movie_actors ORDER BY title
```

The **propertyName** attribute of the **columnType** object can be used to refer to properties of a Cypher entity (either a node or relationship) as shown in the following example:

Listing 44. Defining a view that uses a virtual column (the id function) and an entity returned by the Cypher query

```
[
  {
    "name": "people",
    "query": "MATCH (n:Person) RETURN id(n) AS id, n",
    "columns": [
      {
```

```

    "name": "id",
    "type": "INTEGER"
  },
  {
    "name": "name",
    "propertyName": "n.name",
    "type": "STRING"
  }
]
}
]

```

Which can now be queried as

Listing 45. Querying `v_people`

```
SELECT * FROM v_people WHERE name LIKE 'A%' ORDER BY name
```

As with the `ORDER BY` clause in the `v_movie_actors` example, the predicate and the order clause will be pushed down into the actual Cypher that is passed to the Neo4j DBMS.

Additionally, we support the same format as the "Magnitude Simba Neo4j Data Connector for Business Intelligence Tools" in version 1.0.10 as of January 2022 does. All schema information will however be ignored (schema names and whether a view is hidden or not).

7.4. Restrictions

Cypher-backed views can of course only be read from, not modified (neither updated nor deleted). Cypher-backed views cannot be mixed with regular "tables", that is tables that are treated as labels and then matched on. If you want to query multiple Cypher-backed views at once, you need to enumerate them in the `FROM` clause, such as `view1`, `view2`, `view3` and as appropriate join in the `WHERE` clause with a predicate like this `view1.x = view2.y`. You cannot use the `JOIN`, `LEFT OUTER` or any other `JOIN` clause, as we cannot derive any meaningful relationship from a Cypher-backed view, which we would attempt in such cases.

These restrictions should not matter much, as you can express arbitrary complex expressions inside the query that makes up the view so that your ETL tool or any other program can harness Cypher's full capabilities.

8. Object mapping

8.1. Introduction

There are many different tools to do Object-relational mapping (ORM, O/RM, and O/R mapping tool) with JDBC and the Neo4j JDBC driver does support some of them, such as MyBatis or result-set / tuple oriented mapping with tools such as JDBI or Springs JDBC template. However, many of those tools also generate SQL queries and often times depend on very specific SQL functionality, that we

cannot fully translate in our [SQL to Cypher translation](#), and therefor, your milage may vary.



The most "graphy" ways of Object mapping with Neo4j are either [Neo4j-OGM](#), which is offered with integrations for Spring and Quarkus, or [Spring Data Neo4j](#). Both solutions are built on top of the [common Neo4j Java Driver](#). Those solutions are favorable if you are looking for an end-to-end solution with repository support and advanced mapping and query capabilities.

Sometimes, simple solutions are enough however and one solution that might be already enough is an easy and direct way of mapping graph data to JSON objects and passing back JSON objects into queries. The JDBC spec allows for getting and setting arbitrary objects of arbitrary types via the [ResultSet](#) and [PreparedStatement](#) types and the Neo4j JDBC driver utilizes those for turning nodes and relationships into JSON objects.

As there is no standard JSON object in the JDK, this functionality requires additional, optional dependencies as described in the following sections.

8.1.1. With Jackson Databind

The Neo4j JDBC Driver can utilise [Jackson Databind](#) to transform graph objects into JSON Nodes and read back those objects into maps usable in Cypher queries.

Put the following dependency on your class- or module path to enable mapping into objects of type [JsonNode](#):

Listing 46. Required dependency for Object mapping through Jackson Databind

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.19.1</version>
</dependency>
```

You now can pass [JsonObject.class](#) as type parameter to any overload of [getObject](#) on a [ResultSet](#) that supports a type parameter to retrieve JSON like this:

Listing 47. Retrieving a list of nodes as JSON Array

```
import java.io.StringWriter;
import java.sql.DriverManager;

import com.fasterxml.jackson.core.JsonGenerator;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;

public class ReadNodesIntoJson {

    public static void main(String... args) throws Exception {
```

```

var objectMapper = new ObjectMapper();
objectMapper.enable(SerializationFeature.INDENT_OUTPUT);
objectMapper.enable(JsonGenerator.Feature.IGNORE_UNKNOWN);

try (var connection = DriverManager.getConnection(
"jdbc:neo4j://localhost:7687/movies", "neo4j", "verysecret");
    var stmt = connection.createStatement()) {

    var result = stmt.executeQuery("""
        MATCH (n:Movie) LIMIT 2
        WITH n RETURN collect(n) AS movies
    """);
    result.next();

    var json = result.getObject("movies", JsonNode.class);
    var sw = new StringWriter();
    objectMapper.writeTree(objectMapper.createGenerator(sw), json);
    System.out.println(sw);
}
}

```

The output will look similar to this:

```

[ {
  "elementId" : "4:5c0c7e77-4034-45a1-ab00-a159be8dbf04:0",
  "labels" : [ "Movie" ],
  "properties" : {
    "title" : "The Matrix",
    "tagline" : "Welcome to the Real World",
    "released" : 1999
  }
}, {
  "elementId" : "4:5c0c7e77-4034-45a1-ab00-a159be8dbf04:9",
  "labels" : [ "Movie" ],
  "properties" : {
    "title" : "The Matrix Reloaded",
    "tagline" : "Free your mind",
    "released" : 2003
  }
} ]

```

You'll notice that the nodes carry their element id, a list of labels and a property object. This structure is aligned with the [Query API](#), so that any mapping will be similar to deal with. The JDBC driver however will always use the "Plain JSON" format, so that further mapping into domain objects will be as straight forward as possible, without custom deserializers.

Here's one example that shows a query that structures the result of matching all movies and their actors into maps, collects them as a list, retrieves that again as a JSON node, which can ultimately

mapped by Jacksons Object mapper into a list of domain objects:

Listing 48. Mapping JSON nodes into domain objects

```
import java.sql.DriverManager;
import java.util.List;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public class MapNodesIntoObjects {

    public static void main(String... args) throws Exception {

        var objectMapper = new ObjectMapper();

        record Actor(String name, short born) {
        }
        record Movie(String title, short released, List<Actor> actors) {
        }

        try (var connection = DriverManager.getConnection(
            "jdbc:neo4j://localhost:7687/movies", "neo4j",
            "verysecret"); var stmt = connection.createStatement()) {

            var result = stmt.executeQuery("""
                MATCH (m:Movie)<-[:ACTED_IN]-(a:Person)
                WITH m, collect(a{.*}) AS actors
                ORDER BY m.title
                LIMIT 5
                RETURN collect({title: m.title, released: m.released, actors:
actors})
                """);
            result.next();

            var json = result.getObject(1, JsonNode.class); ①
            var movies = objectMapper.treeToValue(json, new TypeReference<List<Movie>
>() {}); ②

            movies.forEach(System.out::println);
        }
    }
}
```

① First retrieve the list as Json array again

② Use Jacksons `ObjectMapper` to map that array into a list of `Movie` objects containing their actors

Of course, writing back JSON nodes does work, too:

```

import java.sql.DriverManager;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public class WritingObjects {

    public static void main(String... args) throws Exception {

        var objectMapper = new ObjectMapper();

        record Movie(String title, String tagline, long released) {
        }

        var movie = new Movie("title", "tagline", 2025);
        try (var connection = DriverManager.getConnection(
"jdbc:neo4j://localhost:7687/movies", "neo4j",
        "verysecret"); var stmt = connection.prepareStatement("CREATE (m:Movie
$1) RETURN m")) {
            stmt.setObject(1, objectMapper.valueToTree(movie));
            var rs = stmt.executeQuery();
            rs.next();
            var json = rs.getObject("m", JsonNode.class);
            var newMovie = objectMapper.treeToValue(json.get("properties"), Movie
.class);
            System.out.println("New movie " + newMovie + " has id " + json.get(
"elementId"));
        }
    }
}

```

It will produce output similar to this:

```

New movie Movie[title=title, tagline=tagline, released=2025] has id "4:5c0c7e77-4034-
45a1-ab00-a159be8dbf04:173"

```

9. text2cypher

This is an experimental translator inspired by the Neo4j Labs project [text2cypher](#).

If you add this translator to the classpath or use the [text2cypher bundle](#), all queries that start with

```

[]

```

will be treated as *natural language* queries written in **plain english**. The driver will strip the prefix,

and use [OpenAI](#) to translate the input into a Cypher statement. The driver will augment the generation of the query by passing the current graph schema along with the input question.



The following data will be sent to an external API:

- Your database schema, including label names
- Any natural language query

Don't use this translator if you don't want the above, or are not allowed to do so.

This module requires one additional configuration: the OpenAI API key. You can use either a URL parameter, a JDBC property entry, or an environment variable:

- URL parameter/property name is `openAIApiKey`
- Environment variable name is `OPEN_AI_API_KEY`

Listing 50. Example of a valid URL

```
jdbc:neo4j://localhost:7687?openAIApiKey=sk-xxx-your-key
```

Additional configuration properties are

property name	default value
openAIBaseUrl	https://api.openai.com/v1 (defined by langchain4j)
openAIModelName	gpt-4-turbo
openAITemperature	0.0

With that in place, a query such as the following can be translated into Cypher:

```
Q, How was The Da Vinci Code rated?
```

The outcome of the LLM is not deterministic and is likely to vary. While you can execute it directly, we strongly recommend to use `Connection#nativeSQL` to retrieve the Cypher statement, inspect it, and then run it separately. In our test runs, the above questions was most often correctly translated to

```
MATCH (m:`Movie` {
  title: 'The Da Vinci Code'
})<-[r:`REVIEWED`]->(p:`Person`)
RETURN r.rating AS Rating, p.name AS ReviewerName
```

Other times the result was a syntactically correct statement, but it would only return the reviewers and the movie itself. Also note that while a human likely recognizes that you are actually thinking about the average rating, the LLM does not infer this. Making the question more explicit gives

better results:

```
Q, How was The Da Vinci Code rated on average?
```

is translated more accurately to:

```
MATCH (m:'Movie' {
  title: 'The Da Vinci Code'
})<-[:'REVIEWED']-(p:'Person')
RETURN avg(p.rating) AS AverageRating
```



Once a natural language query gets translated into Cypher, the result will be cached and further invocations of that query will use the cached result.

All that statements that do not start with `Q` will be used as-is and treated as Cypher.

Get the full, ready to use bundle here: <https://repo.maven.apache.org/maven2/org/neo4j/neo4j-jdbc-text2cypher-bundle/6.12.3/neo4j-jdbc-text2cypher-bundle-6.12.3.jar>. More information in [Available bundles](#).

Appendix A: Neo4j specific conversions

Neo4j does not support all types used in the relational world. For some of them we offer conversions that are also available in frameworks such as [Spring Data Neo4j \(SDN\)](#). Those conversions are available in this driver as well, so you could use both SDN and this driver interchangeably.

A.1. Data types for fixed-point arithmetic

Neo4j does not support `BigInteger` and `BigDecimal`. The only way to store them is as `String`, and to read them back into the corresponding type. This is in line with SDN and OGM.

Any parameter of those types passed to `PreparedStatement` or `CallableStatement` will be stored as `String`, but can be read back through corresponding methods on the result sets.

A.2. SQL Date, Time and Timestamps

`java.sql.Date`

Maps to Cypher `DATE`

`java.sql.Time`

Maps to Cypher `LOCAL TIME`

`java.sql.Timestamp`

Maps to Cypher `LOCAL DATETIME`

For information on Cypher date types, see [Temporal types](#).

For more precise a mapping, use a Neo4j `Value` instance with the appropriate type and its methods `setObject` and `getObject`.

A.3. Vector support



You need a compatible Neo4j version to use native Vectors with Neo4j.

The Neo4j driver supports the Neo4j `Vector` datatype. The `Vector` type is a uniform container value that has a fixed size of at least one and at most 4096 elements. The latter restriction might be lifted in a future version of Neo4j. Each element is of the same inner type and cannot be null.

The Neo4j `Vector` supports the following inner types:

INTEGER8

Mapping to a Java `byte`

INTEGER16

Mapping to a Java `short`

INTEGER32

Mapping to a Java `int`

INTEGER

Mapping to a Java `long`

FLOAT32

Mapping to a Java `float`

FLOAT

Mapping to a Java `double`

Instances returned by any query, either from a call to the Cypher constructor function `vector()` or by accessing node or relationship properties, can be accessed by using the `getObject` method of the JDBC `ResultSet`, passing in the index or the name of the field as well as the type `org.neo4j.jdbc.values.Vector`. The latter is also the entry point of creating instances from the client side:

`Vector#of(byte[])`

Creates an `INTEGER8` vector (`Int8Vector`)

`Vector#of(short[])`

Creates an `INTEGER16` vector (`Int16Vector`)

`Vector#of(int[])`

Creates an `INTEGER32` vector (`Int32Vector`)

Vector#of(long[])

Creates an **INTEGER** vector (**Int64Vector**)

Vector#of(float[])

Creates an **FLOAT32** vector (**Float32Vector**)

Vector#of(double[])

Creates an **FLOAT** vector (**Float64Vector**)

The type has a guaranteed set of implementations (listed above). Each implementation provides **toArray()** returning a copy of its data as an array of the matching Java primitive. The **Vector** instances themselves are immutable.

Last but not least, vectors returned from any query can also be accessed as **java.sql.Array**.

Appendix B: Neo4j specific syntax for statements

This section deals with conventions for indexing and naming parameters in all types of statements (standard statements, prepared statements and callable statements) and other Neo4j specific syntax.

B.1. Callable statements

B.1.1. General syntax

You can invoke stored procedures as follows:

Common JDBC syntax

```
{? = call db.index.fulltext.queryNodes(?, ?)}
```

Standard Neo4j syntax

```
call db.index.fulltext.queryNodes(?, ?) yield *
```

Enumerating yielded columns

```
{ $propertyName = call db.schema.nodeTypeProperties() }
```

Return (only applicable for functions)

```
RETURN sin(?)
```

B.1.2. Named parameters

Our callable statement implementation (**org.neo4j.jdbc.Neo4jCallableStatement**) does support named parameters. As per the JDBC specification, those are not named placeholders in a query- or statement-string, but the actual, formal parameters for the stored procedures to be called. We support both the **\$** and the colon syntax (i.e. either **\$name** or **:name**).

The assignment `{? = call xxx() }` will be rewritten to `call xxx() yield *`, and `{$x = call xxx() }` will be rewritten to `call xxx() yield x`.

B.1.3. Result sets of callable statements

When you execute a callable statement via `executeQuery`, you must use the result set returned. If you just use `execute`, we assume that the underlying procedure returns only one row and that you use the getters on the statement itself.

Appendix C: Migrating from older versions or other JDBC drivers for Neo4j

There are some other JDBC drivers for Neo4j, under various licenses and with varying features and capabilities. In the following we outline possible migration processes. The basic usage patterns for all JDBC drivers for Neo4j are very similar: in the end, it's "just" about using a JDBC compliant driver, and you would use it as described in the original [Java tutorial about JDBC](#).

C.1. Migrating from version 4 or 5 of this driver

Version 5 and 4 of the Neo4j JDBC Driver have been mainly developed by [Larus BA, Italy](#), a certified consulting and integration solutions partner for Neo4j.



The most important change that you need to make is removing the dependency on `org.neo4j:neo4j-jdbc-bolt`. You need to replace it with `org.neo4j:neo4j-jdbc` or one of the bundles we provide, see [available bundles](#).

C.1.1. URL format, parameters and main driver class

The previous version mapped the URLs from the common Java driver onto `jdbc:`, i.e. using `jdbc:neo4j:neo4j+s://foobar.io:7687/` for connecting against a database running on host `foobar.io` on port 7687.

The following URLs (direct bolt connection) are not supported:

- `jdbc:neo4j:bolt://<host>:<port>/`
- `jdbc:neo4j:bolt+s://<host>:<port>/`
- `jdbc:neo4j:bolt+ssc://<host>:<port>/`

The following URLs behave the same but must be rewritten:

- `jdbc:neo4j:neo4j://<host>:<port>/` becomes `jdbc:neo4j://<host>:<port>/`
- `jdbc:neo4j:neo4j+s://<host>:<port>/` becomes `jdbc:neo4j+s://<host>:<port>/`
- `jdbc:neo4j:neo4j+ssc://<host>:<port>/` becomes `jdbc:neo4j+ssc://<host>:<port>/`

The following configuration properties are **not** supported and have no replacement:

- `leaked.sessions.logging`
- `readonly`
- `usebookmarks`
- `max.transaction.retry.time` (this driver has no built-in retry mechanism)

As with any persistent database connection you want to cater for failed transactions. We made good experience with [resilience4j](#) which fits in well with common Java frameworks, such as Spring Boot.

The following properties can be achieved with standardized JDBC settings:

- `encryption` — Use the appropriate transport scheme (`neo4j`, `neo4j+s` or `neo4j+ssc`)
- `autocommit` — Use `java.sql.Connection.setAutoCommit`

Connection pooling can be achieved with any JDBC-compliant connection pool. The following properties have no effect:

- `max.connection.lifetime`
- `max.connection.poolsize`

The following properties just have different names:

- `connection.acquisition.timeout` is `timeout` (as query-parameter to the URL or inside the configuration properties)

The following properties can be achieved using a different URL:

- `database` is now part of the URL: instead of specifying `jdbc:neo4j:neo4j+s://foobar.io:7687?database=abc` you would use the database name as path segment in the url, such as: `jdbc:neo4j+s://foobar.io:7687/abc`

If your tooling requires to use a concrete driver class, this JDBC driver has only `org.neo4j.jdbc.Neo4jDriver`. If you depend on a `javax.sql.DataSource`, we provide `org.neo4j.jdbc.Neo4jDataSource`.

C.1.2. Flattening

While the Neo4j JDBC Driver does not support the `flatten` option, it can emulate its effect. `flatten` un-nests returned nodes and relationships by providing all their properties as individual columns.

If you enable automatic [SQL to Cypher translation](#), any `*-select` will inspect whether it affects nodes or relationships and will un-nest their properties, so that a `SELECT * FROM Movie m` will effectively become `MATCH (m:Movie) RETURN m.title AS title, m.released AS released`. For more information, see [Star-Selects](#).

In case you want to access the actual node, return the whole table alias or just use Cypher.

The Neo4j JDBC Driver supports complex object types as return types.